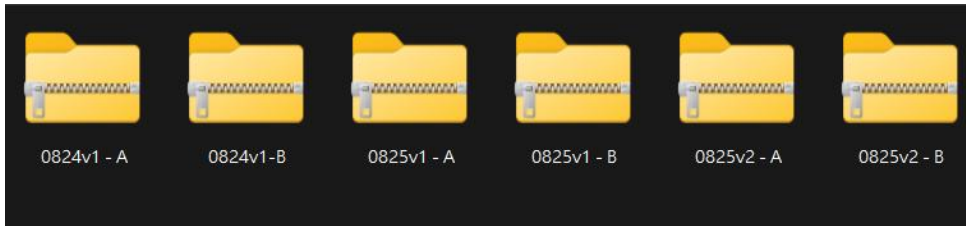


Introduction to Git & GitHub

Why Version Control?

Many zip files with vague names



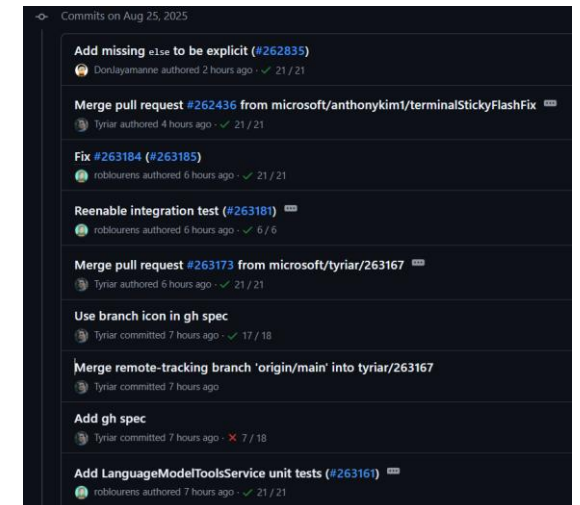
Overwrites when sharing via USB / chat



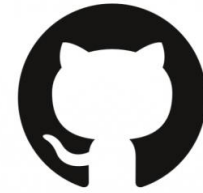
Unclear division of work
No safe way to experiment in parallel



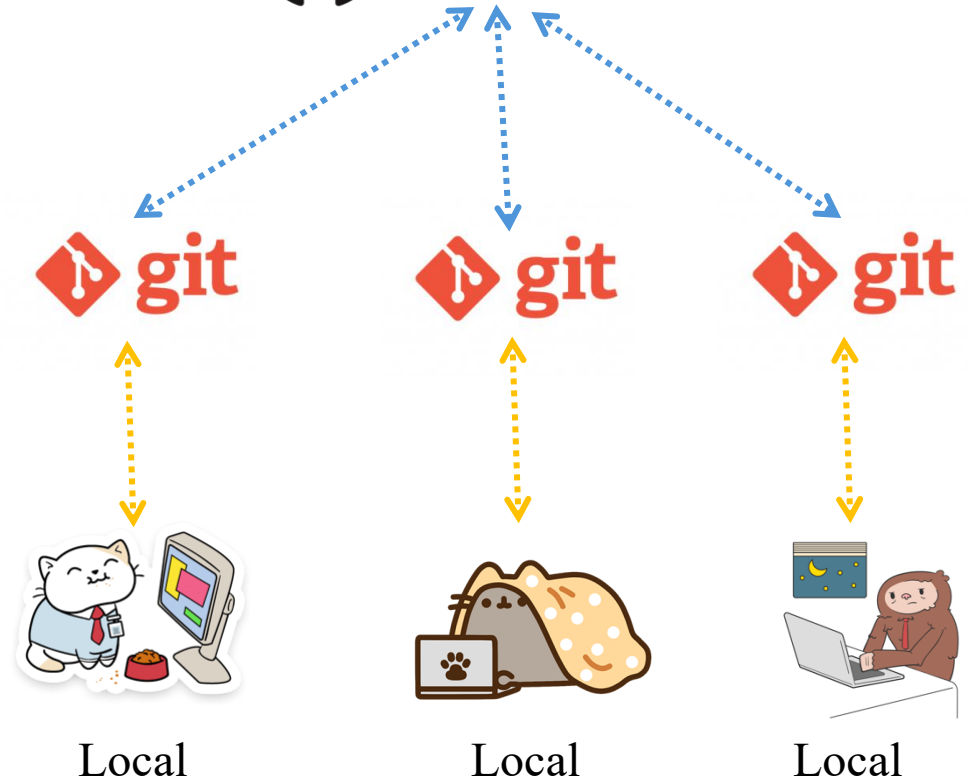
Git enables: history, authorship, parallel branches, safe merges, quick rollback.



Git vs GitHub



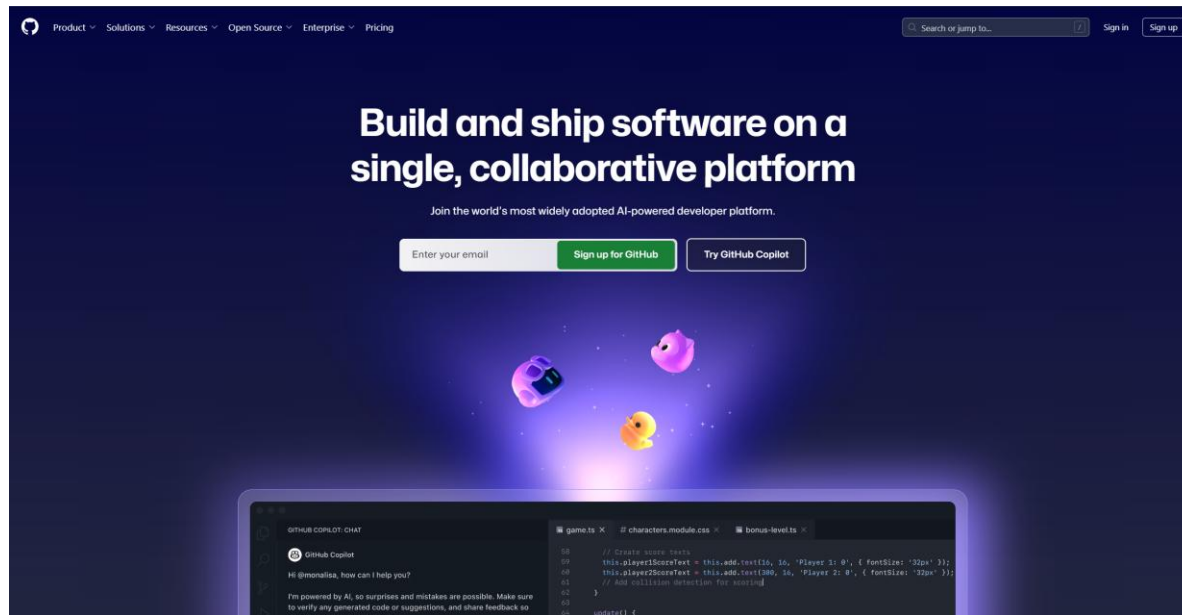
Essence	Distributed Version Control System (software tool)	Cloud hosting platform built on top of Git
Installation	Installed on your local machine	Web service, no installation required
Core Functions	Version control, branching, merging	Remote hosting, collaboration, Issues, Pull Requests
Usage Mode	Git: Command line or GUI clients	Web interface + Git commands
Offline Use	✅ Fully usable offline	❌ Requires internet access
Analogy	Like Word's Track Changes feature	Like a shared folder on Google Drive
Competitors	SVN, Mercurial	GitLab, Bitbucket



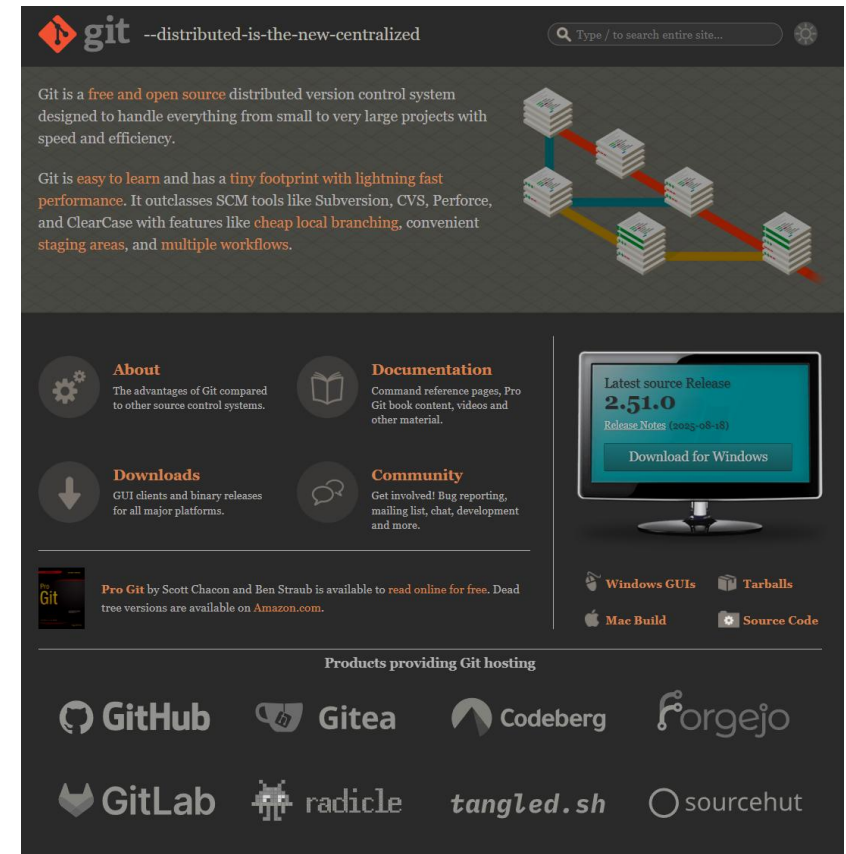
Installation

Pre-Class Setup

[Link: GitHub](#)



[Link: Git](#)



Create a GitHub account

[Link: GitHub](#)

1. Sign up with email / Google / SSO

Sign up for GitHub



pr

Email*

Email

Password*

Password

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

Username

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region*

Taiwan

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

☐ Receive occasional product updates and announcements

Create account >

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

2. Pick a stable username
(affects repository URLs)

Sign up for GitHub

Email

[Use a different Google account](#)

Username*

Username

Your Country/Region*

Taiwan

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

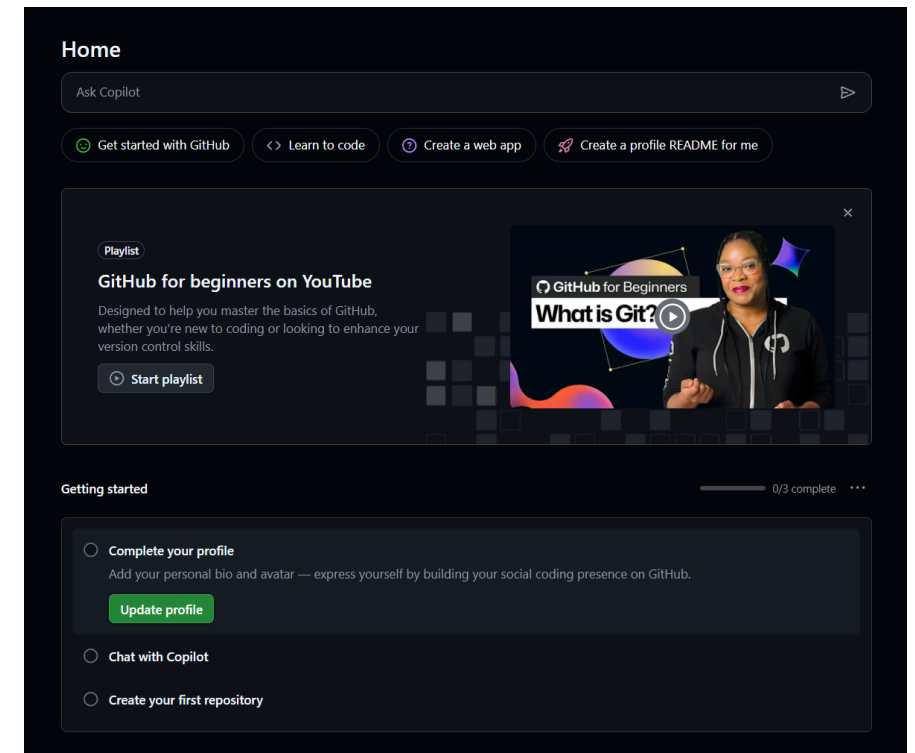
Email preferences

☐ Receive occasional product updates and announcements

Create account >

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

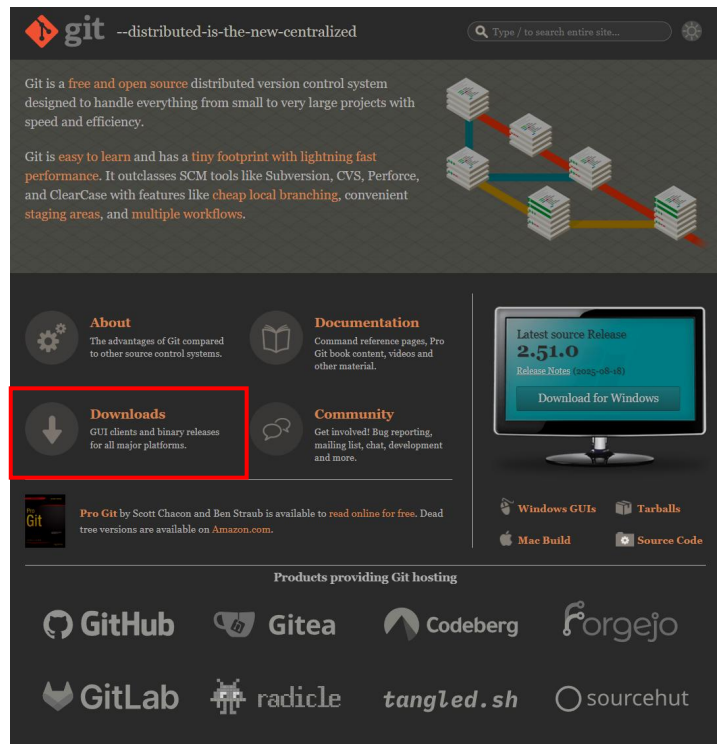
3. Verify email to enable full features



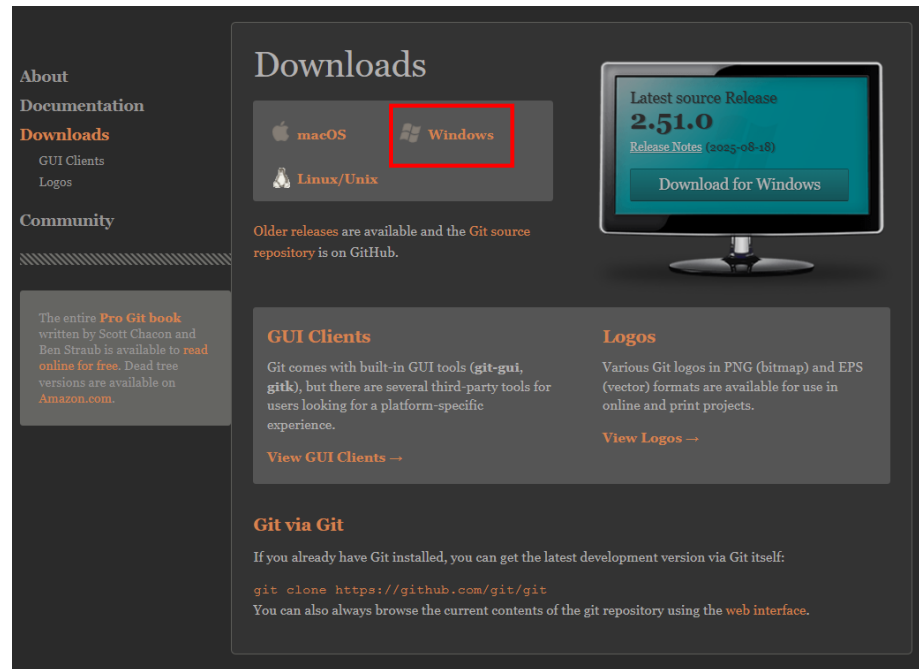
Download Git

[Link: Git](#)

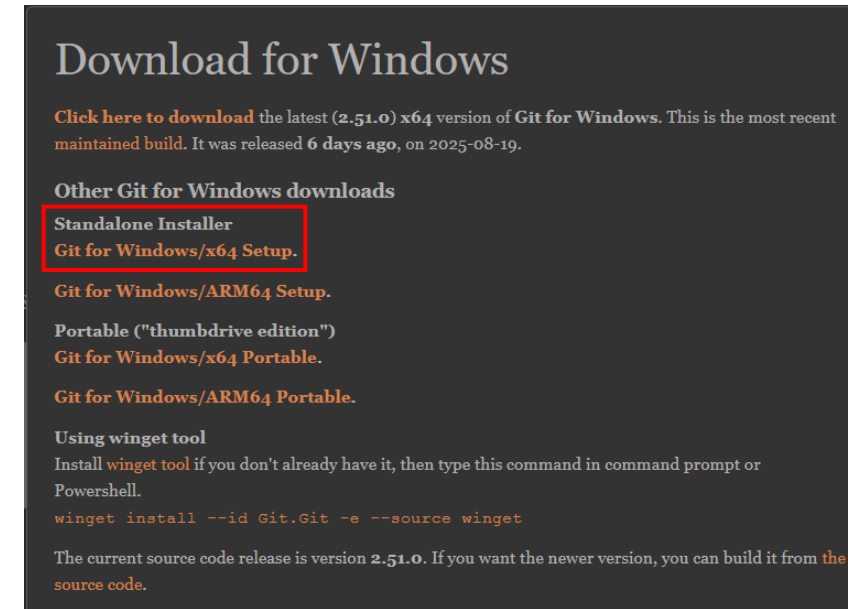
1. Visit official site



2. Select your platform (macOS 、Linux use command)



3. Download latest stable build

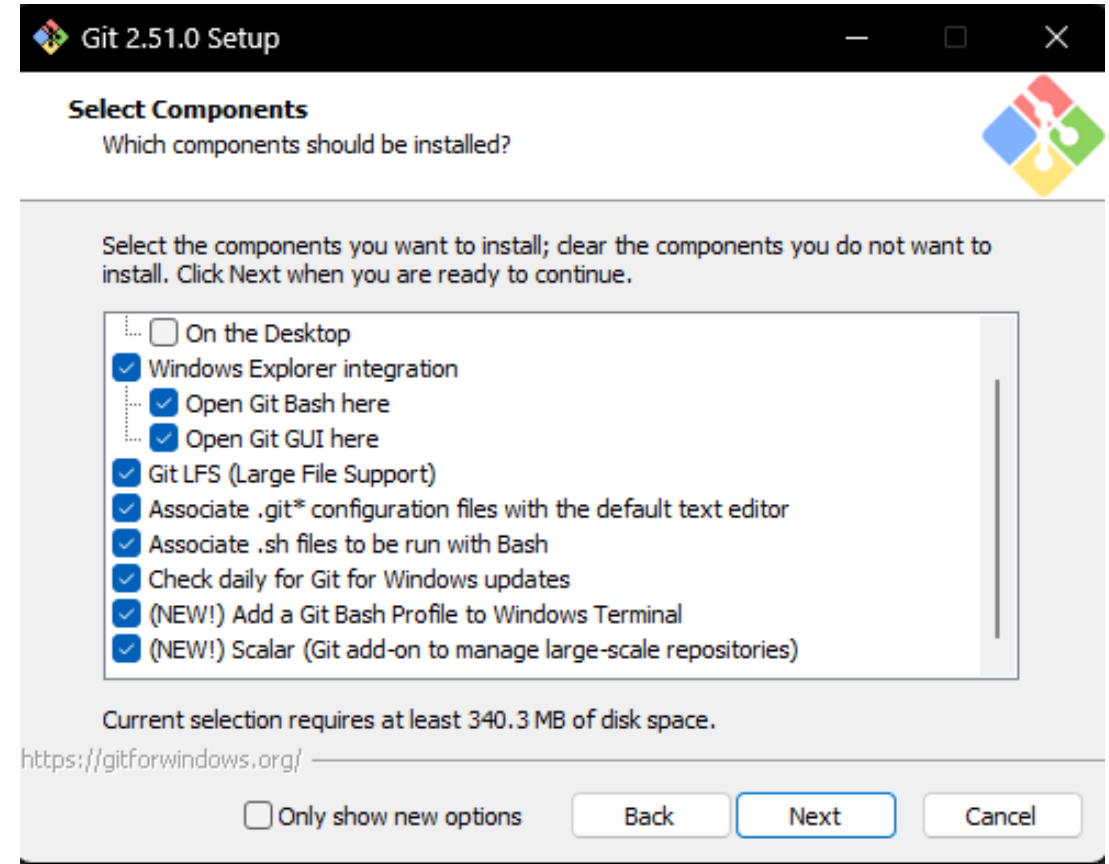


Next



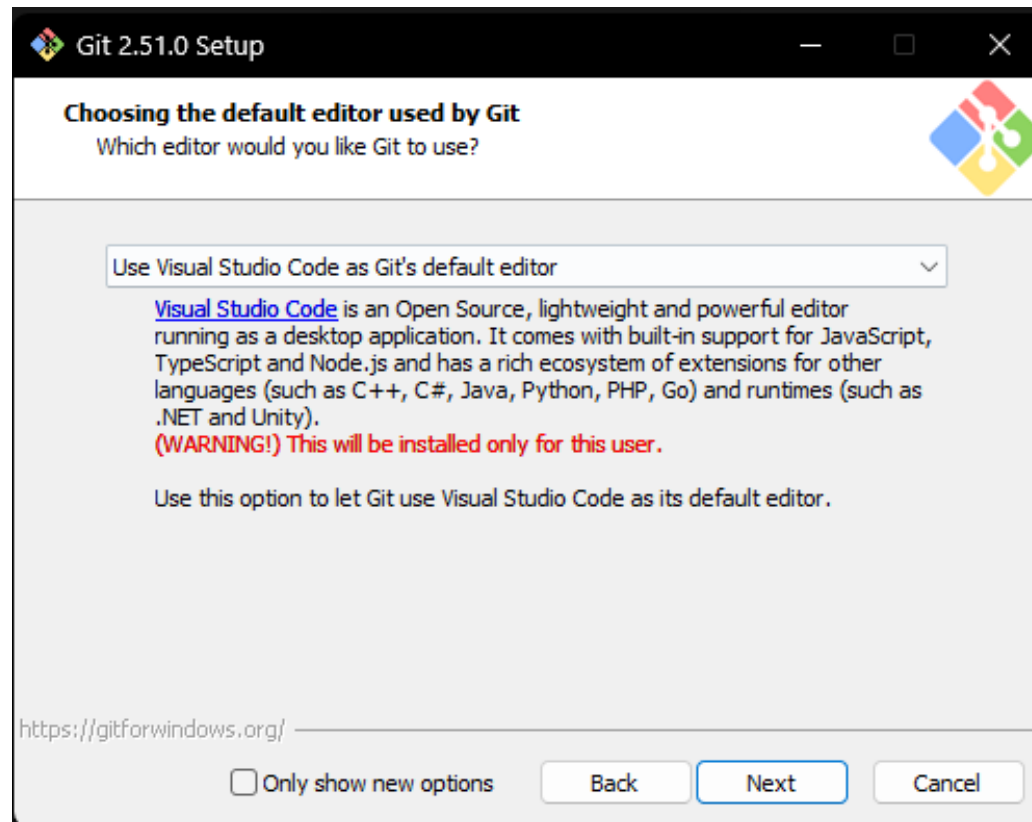
Usually no need to change

- Required: Windows Explorer integration (right-click to open Git)
- Recommended: Open Git Bash here, Add Git Bash Profile to Windows Terminal
- Optional: Open Git GUI here (graphical interface)
- For large repositories: Git LFS (can be installed later if needed)
- Convenient: Associate .git*, Associate .sh (allows configuration files/scripts to be used directly in an editor/Bash)
- Scalar (required for large repositories)



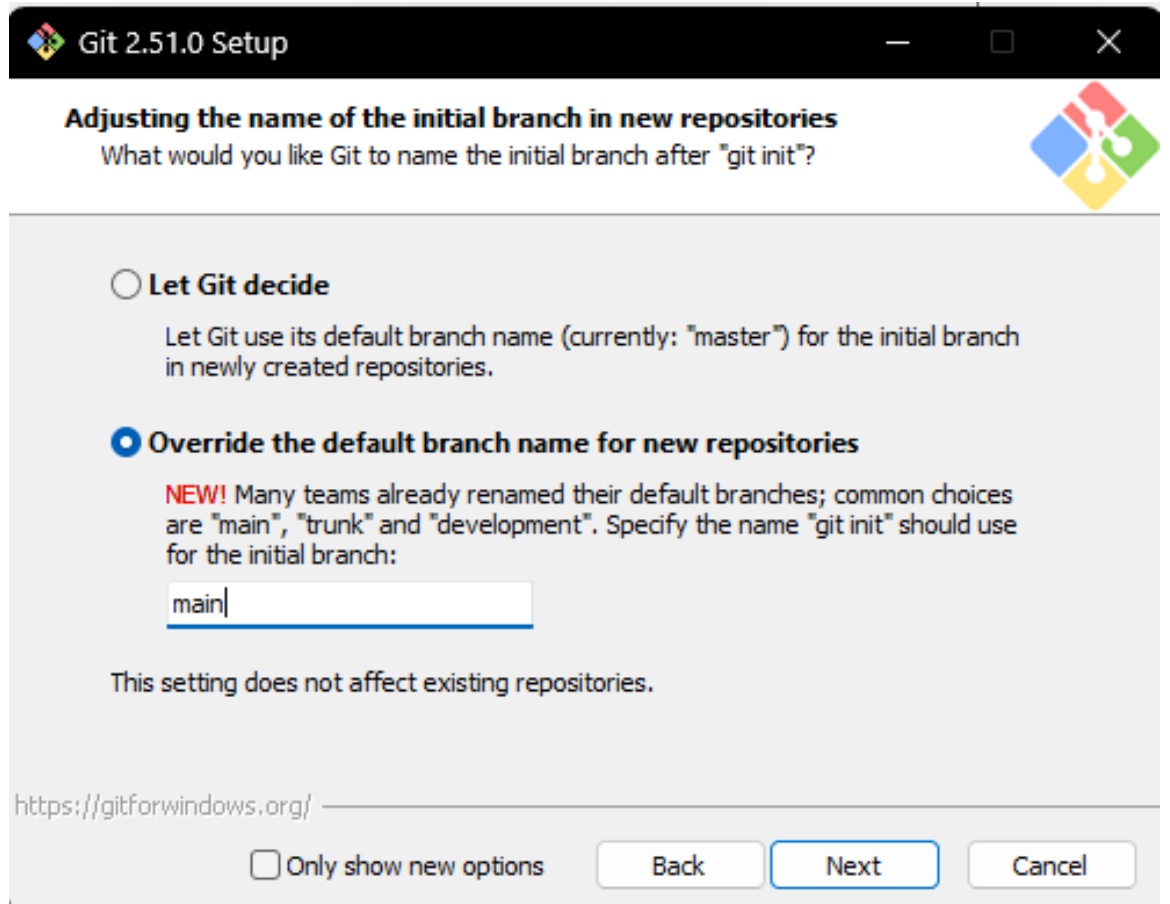
Choose a common editor

- First choice : Use Visual Studio Code as Git's default editor



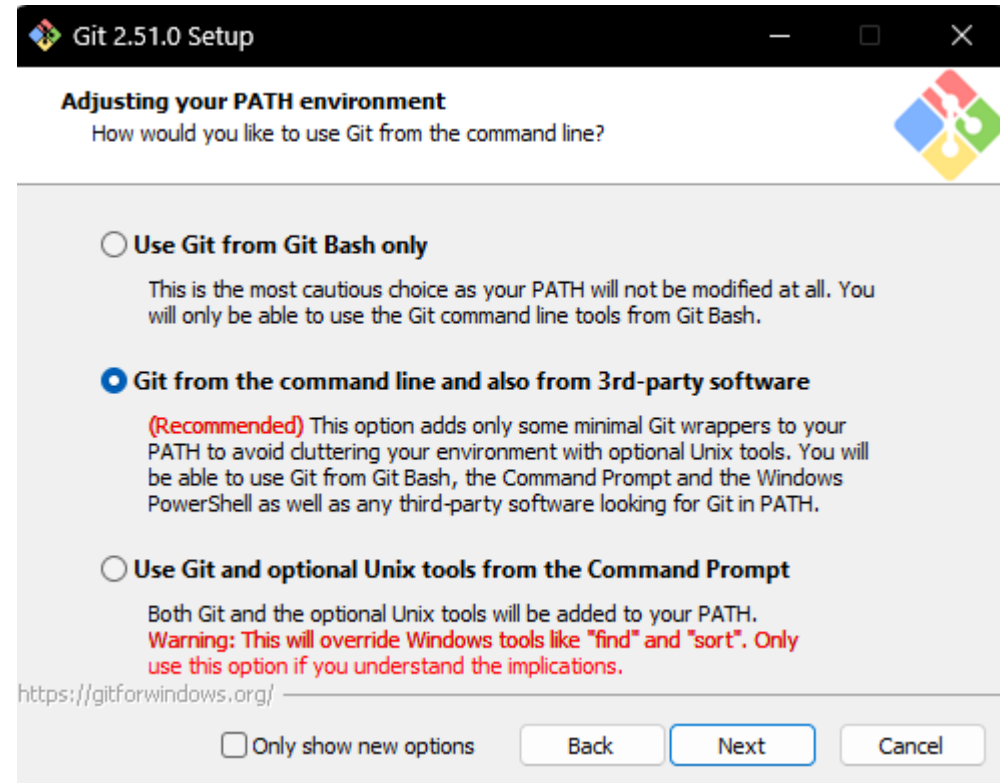
Initial branch name

- Select the second one.
- The main branch is now usually called “main”.
- This is consistent with GitHub's default behavior and avoids confusion between master and main.

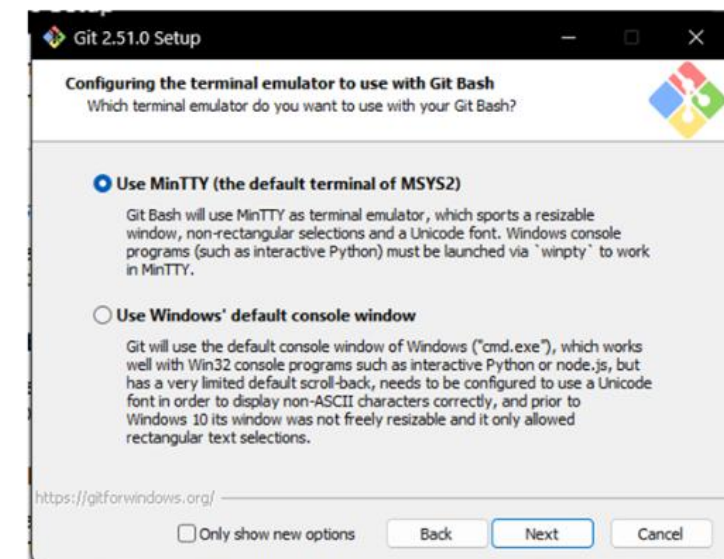
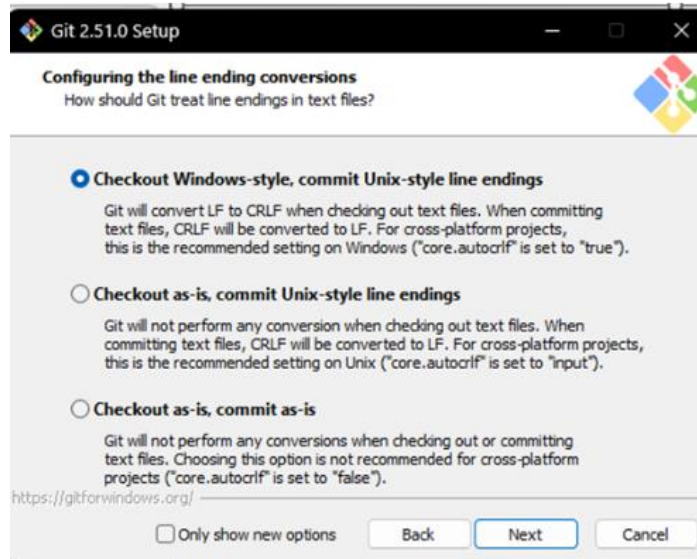
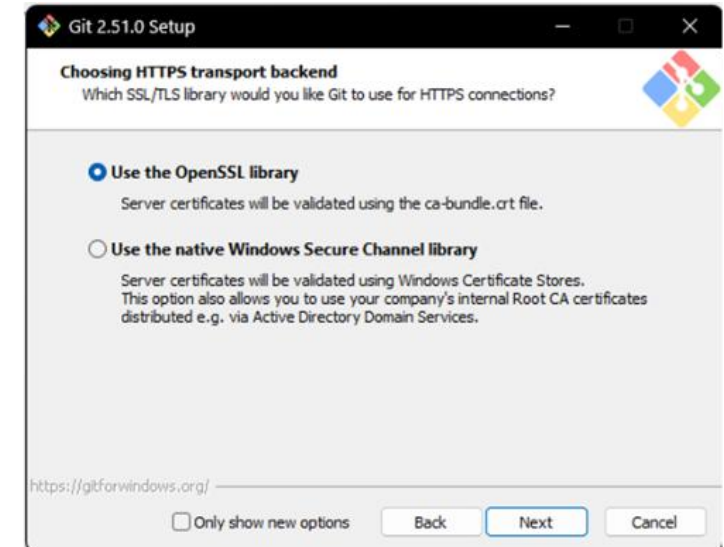
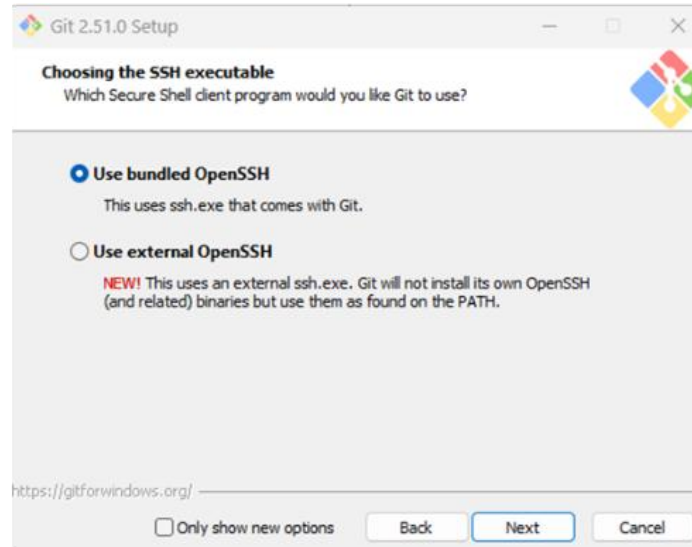


Environmental variables

- Usually the middle one is good

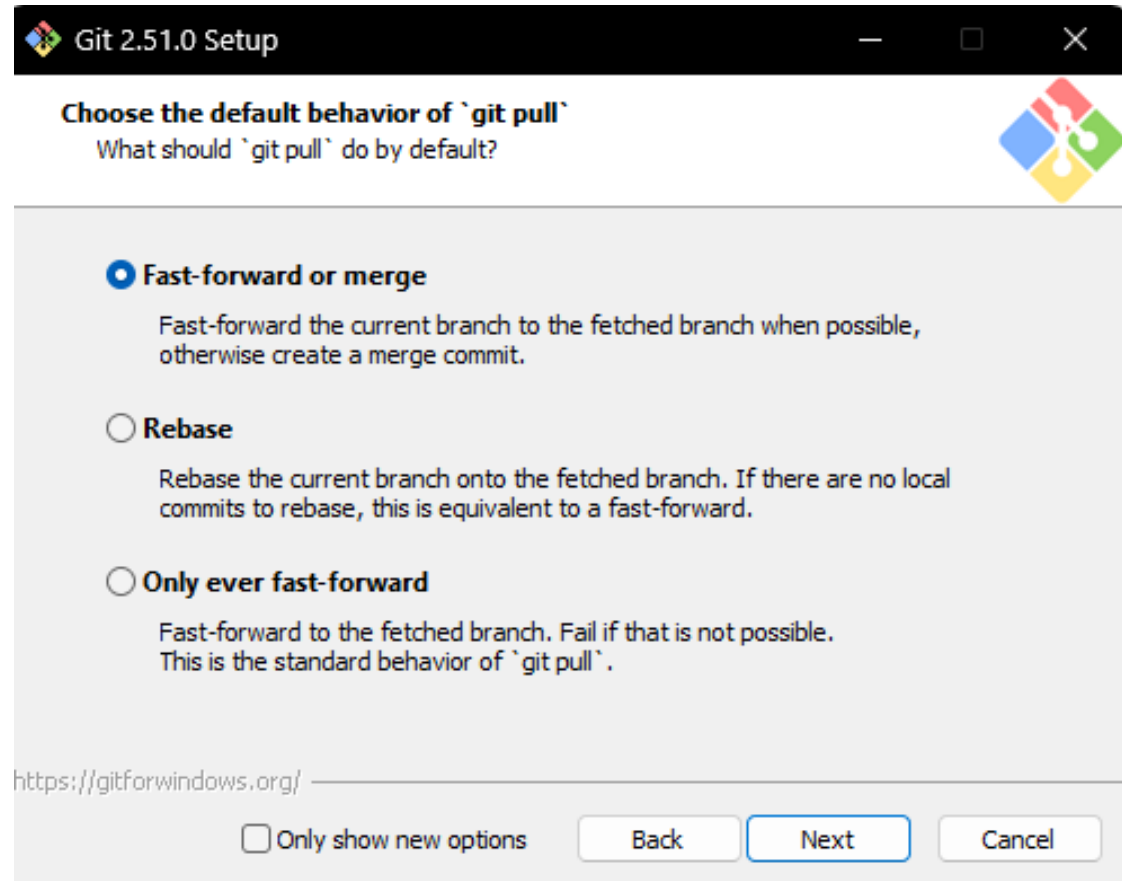


Just use the default

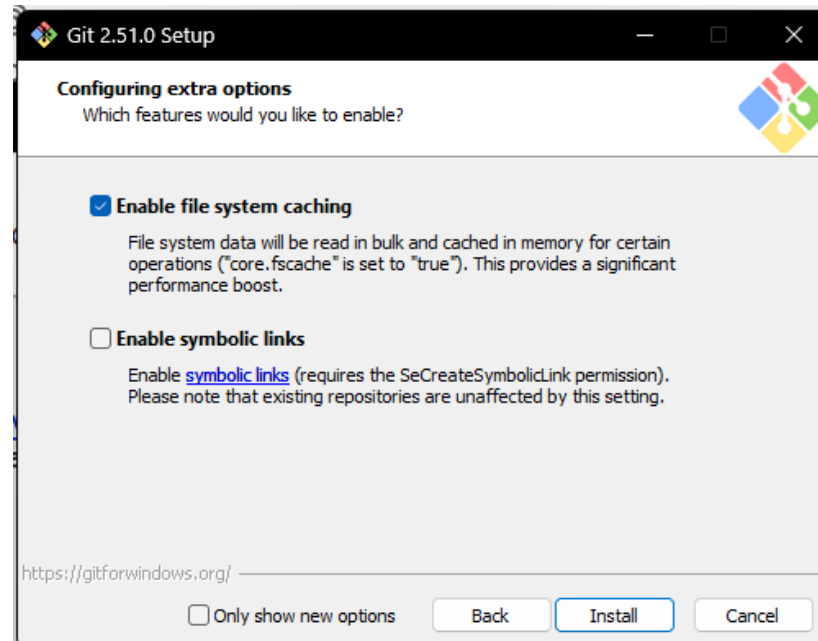
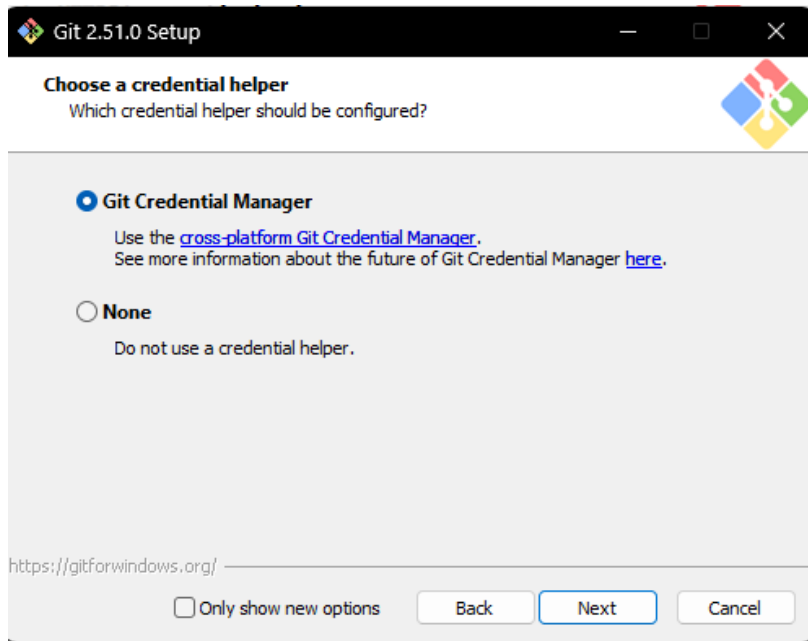


git pull default behavior

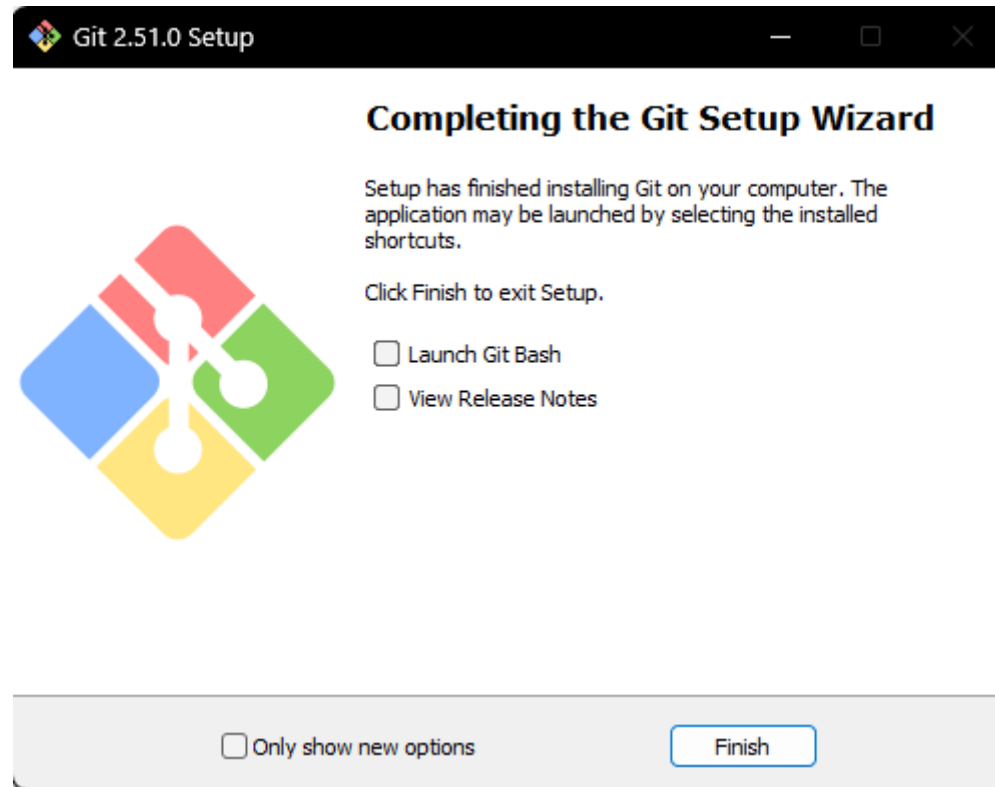
- Recommended: Fast-forward or merge (default; preserves true history)
- Rebase: Provides linear history but more advanced concepts
- (Suitable for those who are already familiar with the process)
- Only ever fast-forward: Will fail if a merge is required; not recommended for beginners



Continue to press Next

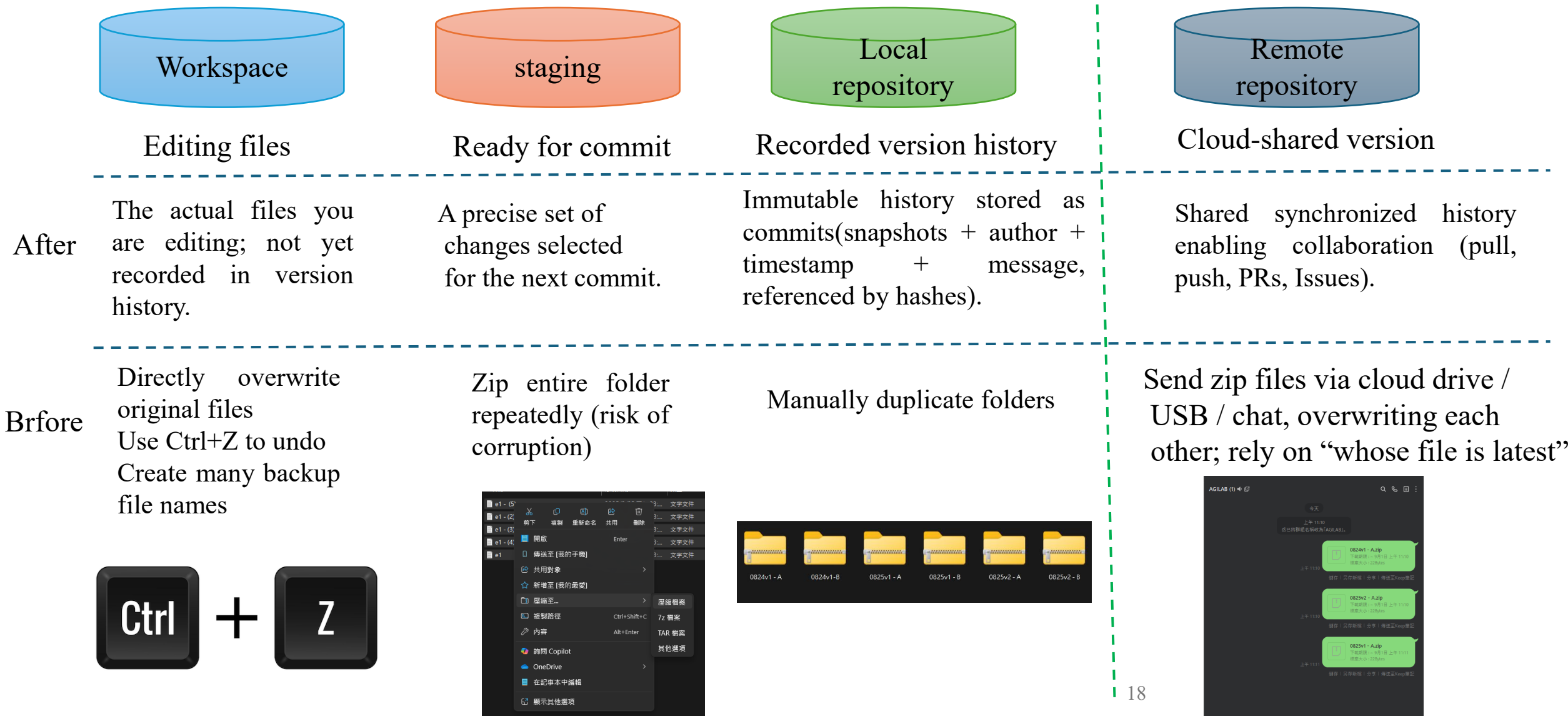


Finish

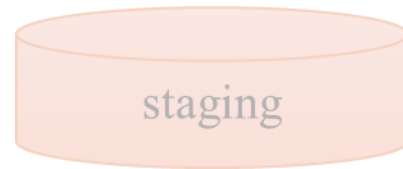


Implementation

Workspace、staging、 Local repository、Remote repository

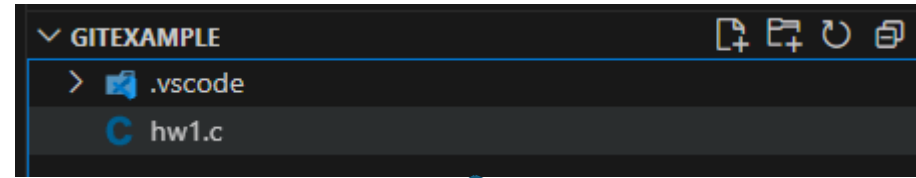


Initial



Initialize the repository

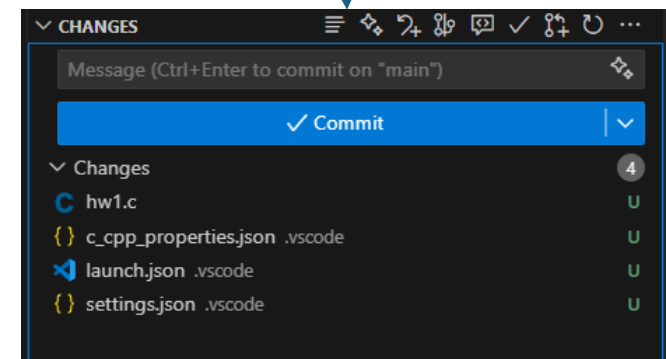
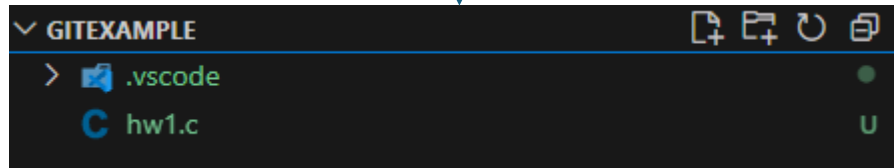
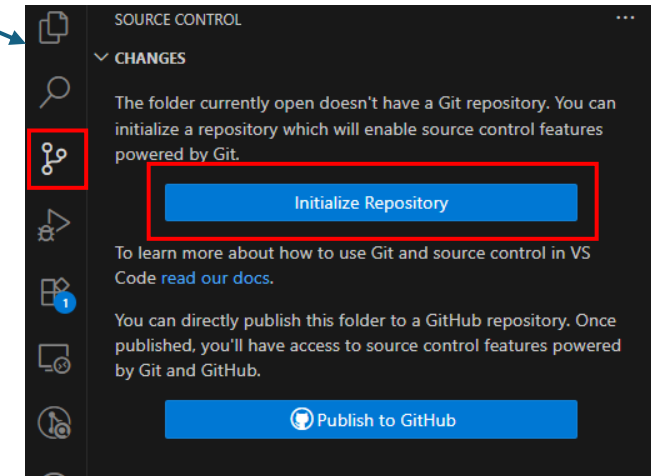
Create a file



Use command

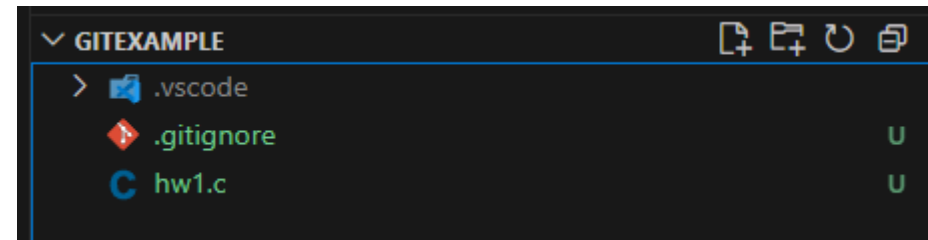
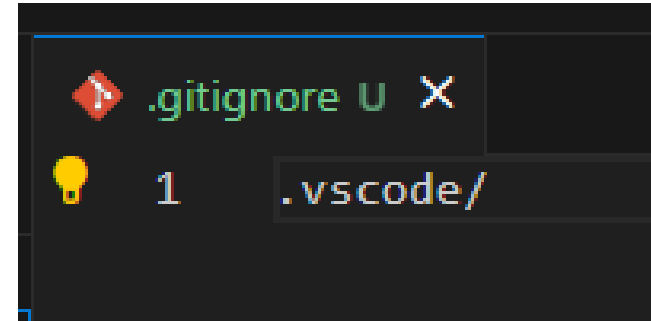
```
E:\GitExample>git init  
Initialized empty Git repository in E:/GitExample/.git/
```

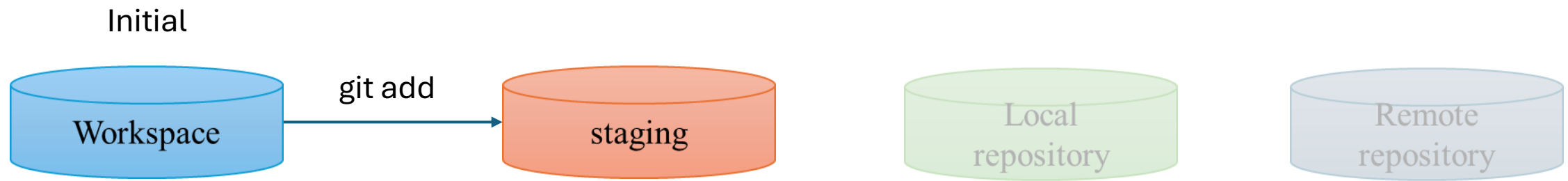
Use VS code UI



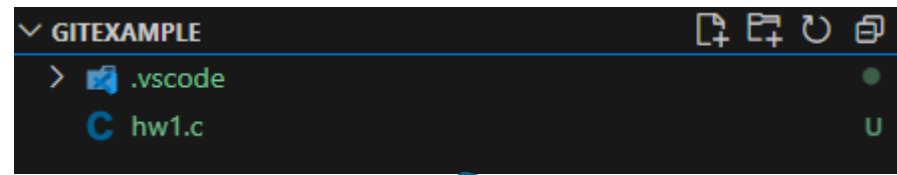
.gitignore

Let Git ignore files or folders that no longer need to be tracked. For example: configuration files, keys, repositories...



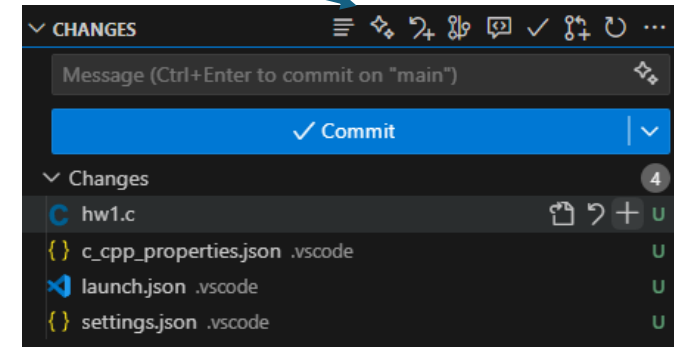


git add git status



git add files

```
E:\GitExample>git add hw1.c
```



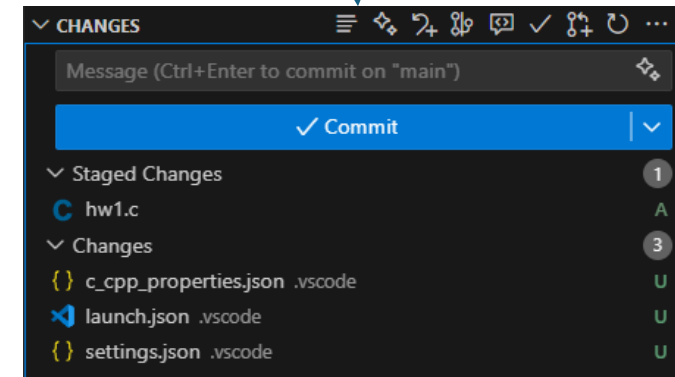
git status

```
E:\GitExample>git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hw1.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .vscode/
```



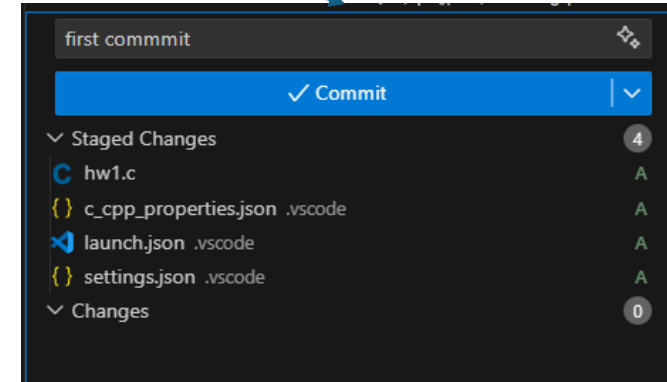


git commit

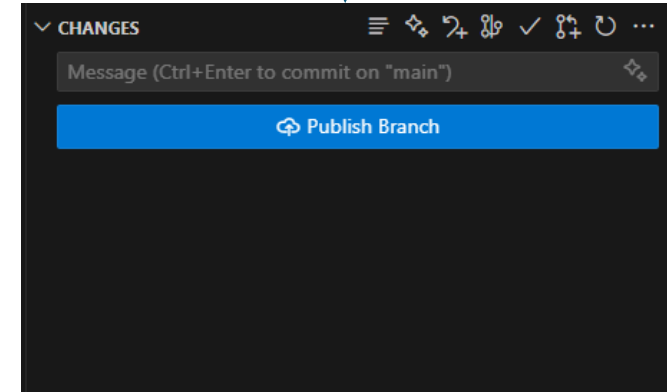


git add .
git commit -m "message"

```
E:\GitExample>git add .  
  
E:\GitExample>git commit -m "first commit"  
[main (root-commit) 0a3f7da] first commit  
2 files changed, 1 insertion(+)  
create mode 100644 .gitignore  
create mode 100644 hw1.c
```

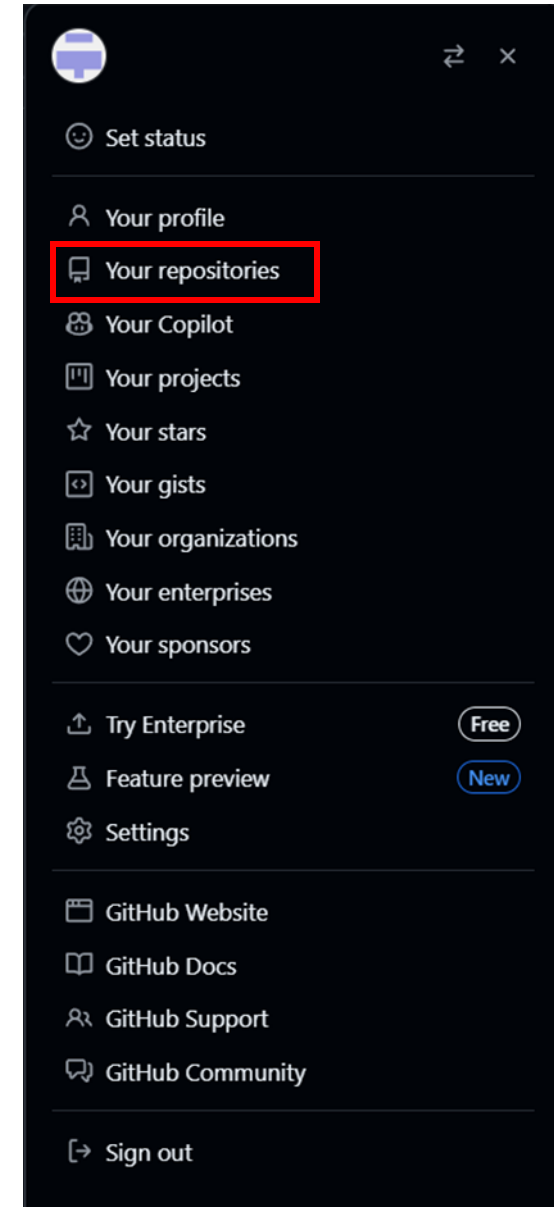
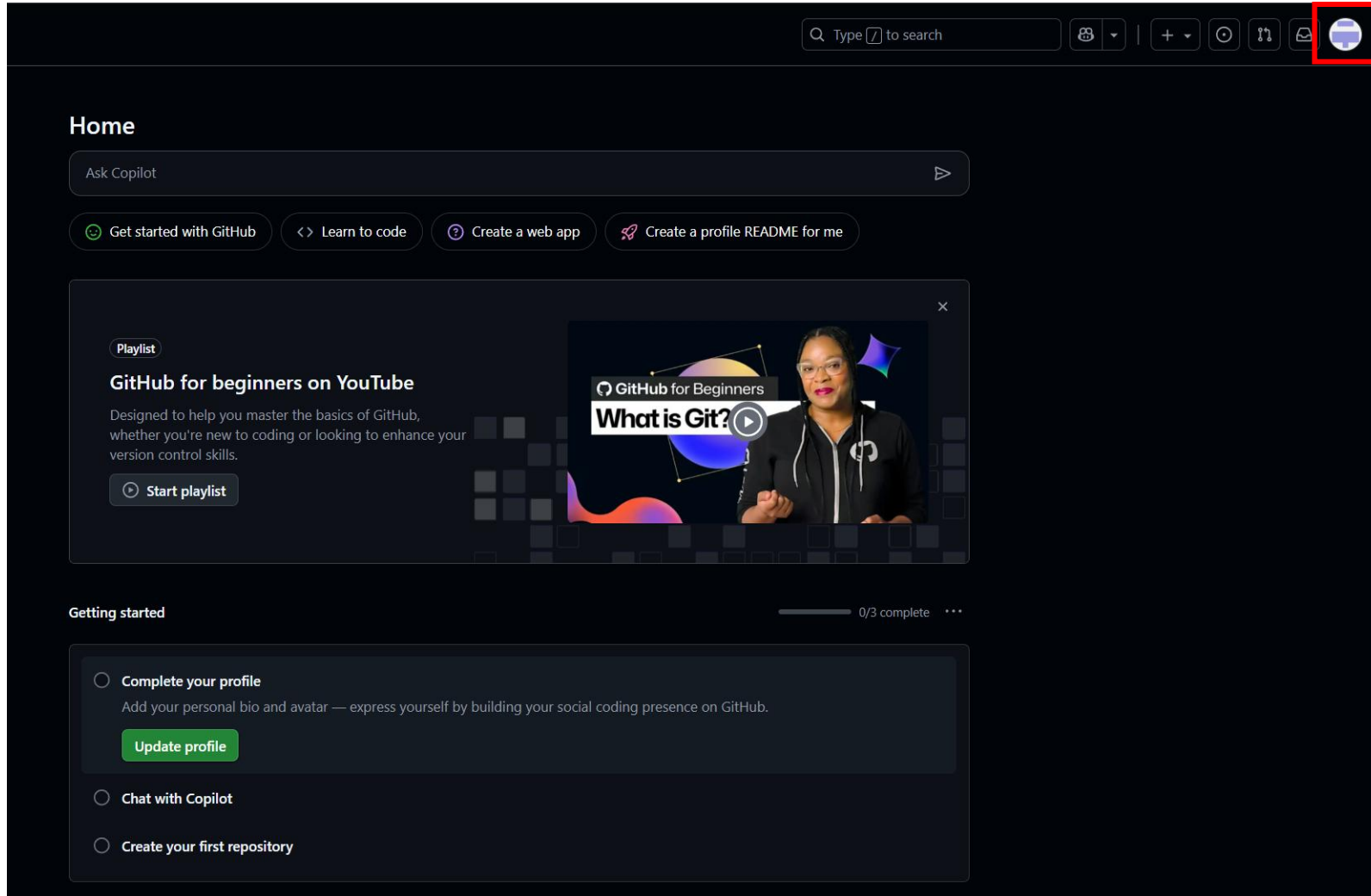


```
E:\GitExample>git status  
On branch main  
nothing to commit, working tree clean
```





Creating a remote repository



Creating a remote repository

Repository name: Try not to change it.
Description: Optional.
Visibility: Private or public.
README: Create it locally.
.gitignore: Create it locally.
License: Create it locally.

Find a repository... Type Sort **New**

1 General

Owner * / Repository name *
example
✓ example is available.

Great repository names are short and memorable. How about [urban-octo-palm-tree?](#)

Description
0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#) No license

Create repository

git add remote

After creating the connection, we need to connect to the local and remote servers.

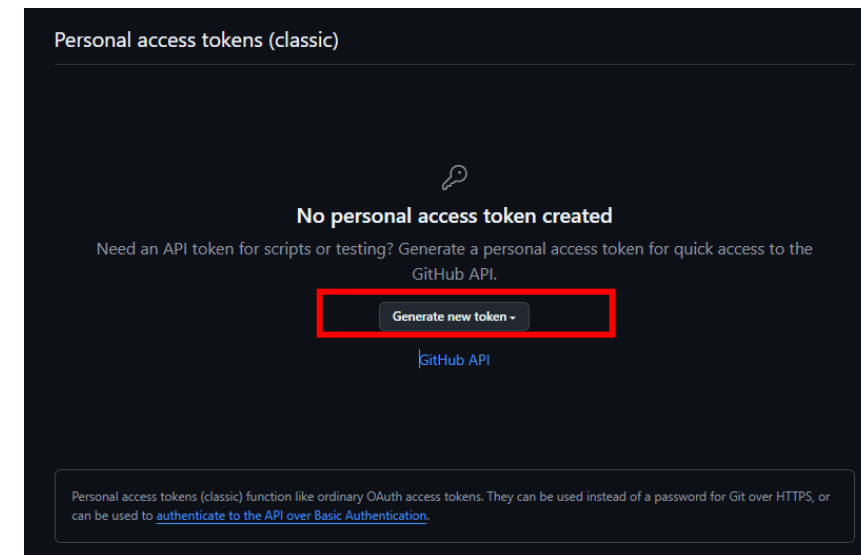
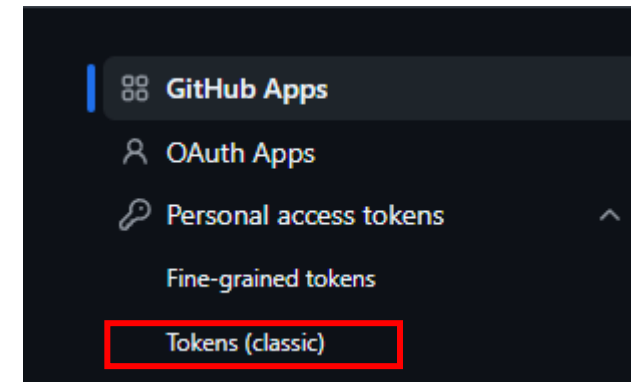
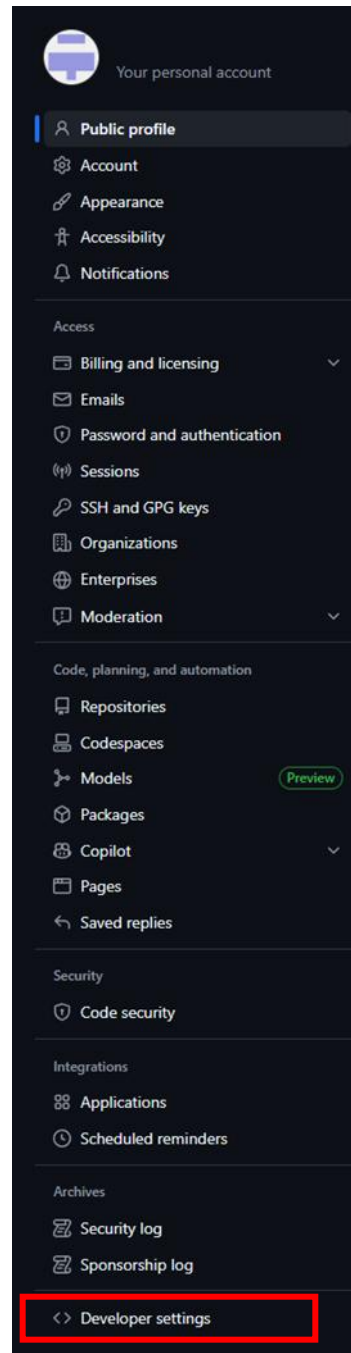
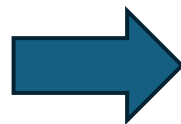
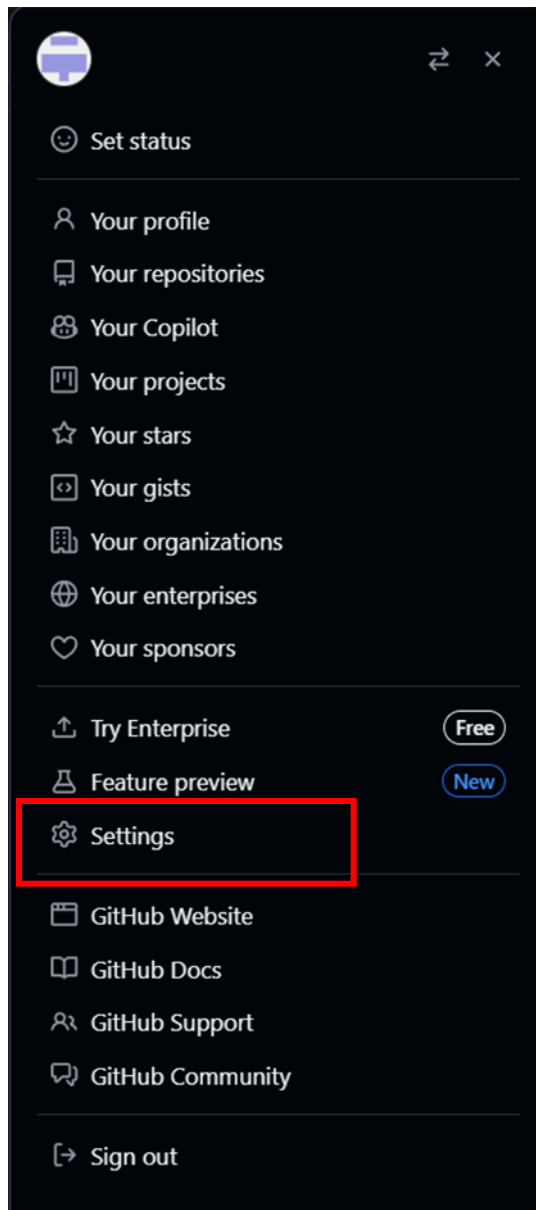
There are two ways to do this:

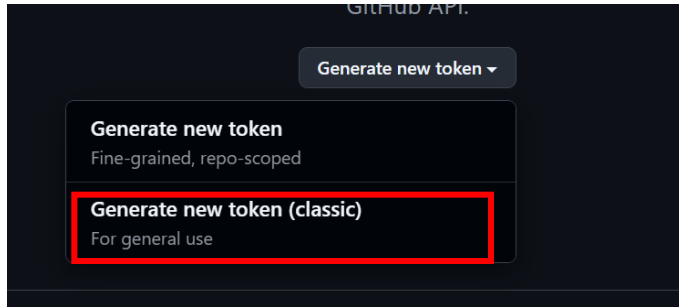
Personal laptop/long-term development:
SSH

Public classroom computer or temporary
operation: HTTPS

Aspect	HTTPS	SSH
Authentication Method	Username + Personal Access Token (PAT)	Public / private key pair (public key uploaded to GitHub, private key kept locally)
Initial Setup Complexity	Low (create a PAT and paste it once)	Medium (generate keypair, optional passphrase, upload public key)
Convenience After Setup	May need credential caching; occasional re-entry required	Once set up, usually no further prompts (unless passphrase)
Typical Use Case	Temporary / shared / classroom machines (do not leave a private key behind)	Personal long-term development environment
Risk Point	PAT leak or loss (can be revoked)	Private key compromise (must immediately remove key)
Permission / Scope Control	PAT scopes can be fine-grained	A single user key often grants access to all repos of that user (unless using per-repo deploy keys)
URL Format	https://github.com/USER/REPO.git	git@github.com:USER/REPO.git







New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

example

What's this token for?

Expiration

📅 7 days (Sep 01, 2025) ▾

The token will expire on the selected date

Select scopes

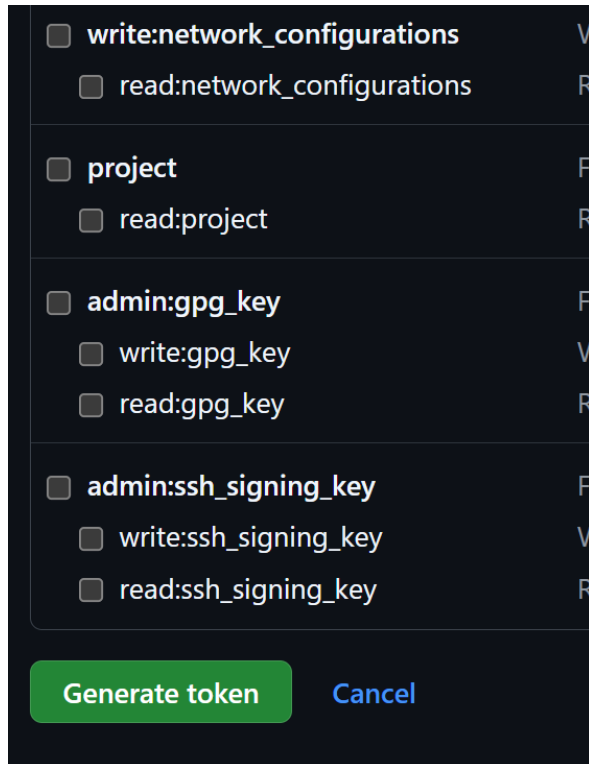
Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☐ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☐ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☐ security_events	Read and write security events

Note: Write a name that's easy for you to understand.

In the permissions settings, just check "repo" first.

Press Generate token

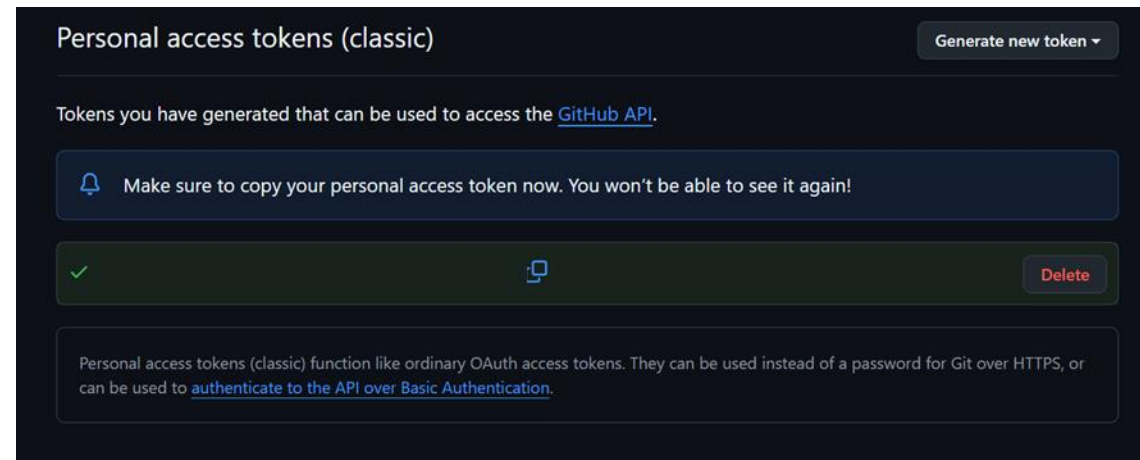


A dark-themed dialog box for generating a GitHub token. It features a list of permissions with checkboxes. The permissions are grouped into four sections: network configurations, project, gpg key, and ssh signing key. Each section has a main category and a sub-category. At the bottom, there are two buttons: a green 'Generate token' button and a blue 'Cancel' button.

- ☐ write:network_configurations
- ☐ read:network_configurations
- ☐ project
- ☐ read:project
- ☐ admin:gpg_key
- ☐ write:gpg_key
- ☐ read:gpg_key
- ☐ admin:ssh_signing_key
- ☐ write:ssh_signing_key
- ☐ read:ssh_signing_key

Generate token Cancel

Copy the generated token, which will be used next



A dark-themed page titled 'Personal access tokens (classic)'. It has a 'Generate new token' button in the top right. Below the title, it says 'Tokens you have generated that can be used to access the [GitHub API](#).' There is a notification box with a bell icon and the text 'Make sure to copy your personal access token now. You won't be able to see it again!'. Below that is a list of generated tokens, each with a green checkmark, a copy icon, and a 'Delete' button. At the bottom, there is a text box explaining that these tokens function like ordinary OAuth access tokens and can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Personal access tokens (classic) **Generate new token**

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

✓ [Copy Icon] Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Adding a new remote repository

```
git remote add origin https://github.com/<user>/example.git
```

Add a new remote repository

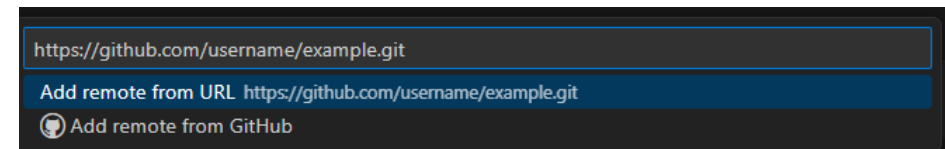
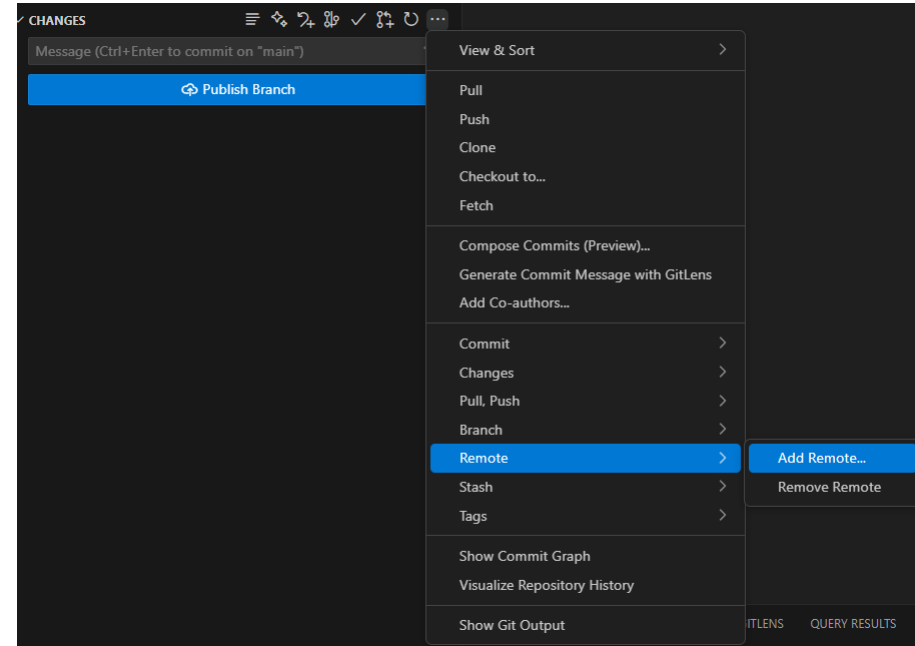
```
git branch -M main
```

Switch the default branch name (if main is the default branch name when installing Git, you can skip this step).

```
C:\Users\Student\example>git remote add origin https://github.com/<user>/example.git
```

```
C:\Users\Student\example>git branch -M main
```

VS Code GUI



git push

git push -u origin main
Push updates to the remote

```
C:\Users\Student\example>git push -u origin main
Username for 'https://github.com': <user>
Password for 'https://<user>@github.com':
```



成功

```
C:\Users\Student\example>git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 240 bytes | 240.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/<user>/example.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

The first time you push, you'll be asked to enter:

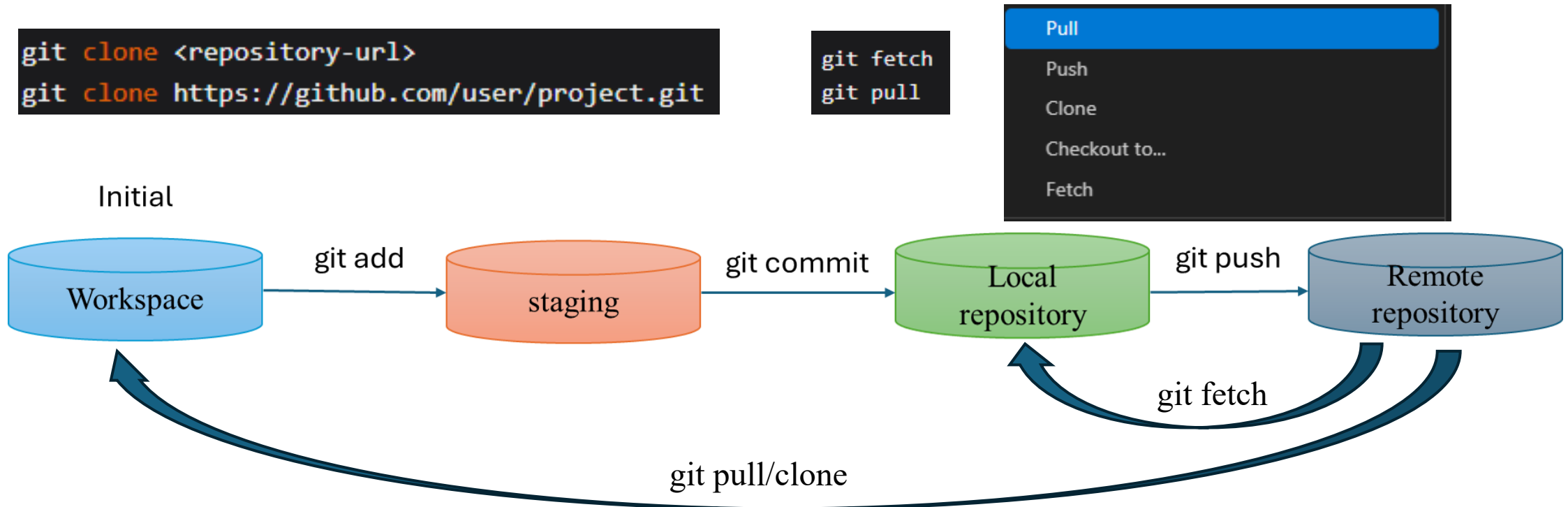
Username: Your GitHub username

Password: This is not your GitHub password, but your Personal Access Token (PAT)



clone & pull

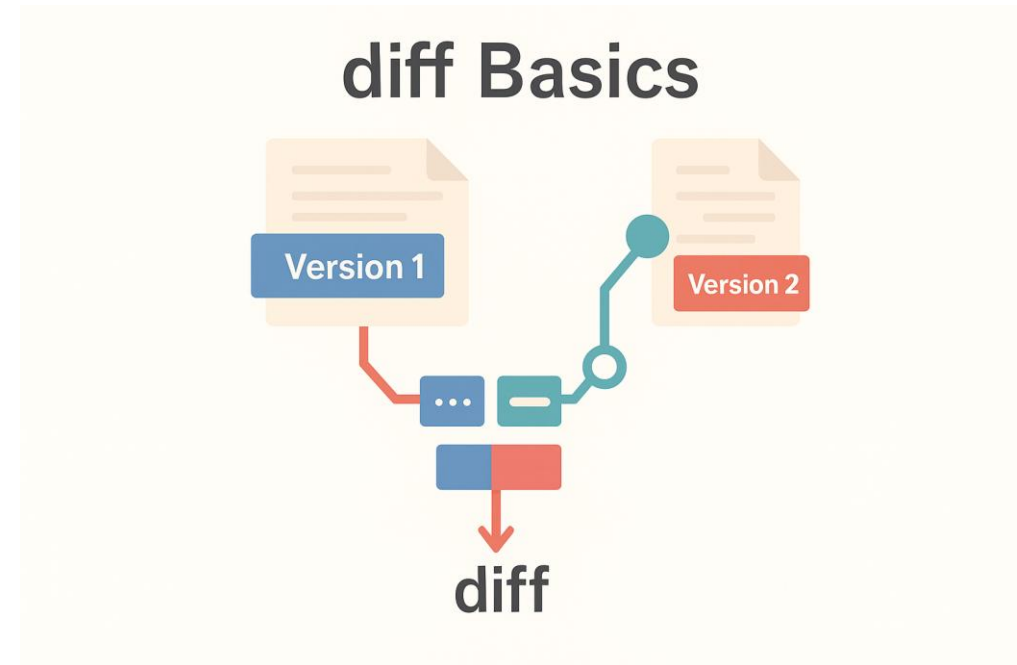
- git clone: First-time retrieval of the entire remote repository.
Creates the project folder, initializes .git, downloads full history, sets the origin remote.
- git pull: Used after the local repository exists to update the current local branch.
- Clone is needed once; pull is a repeated everyday operation.



diff Basics

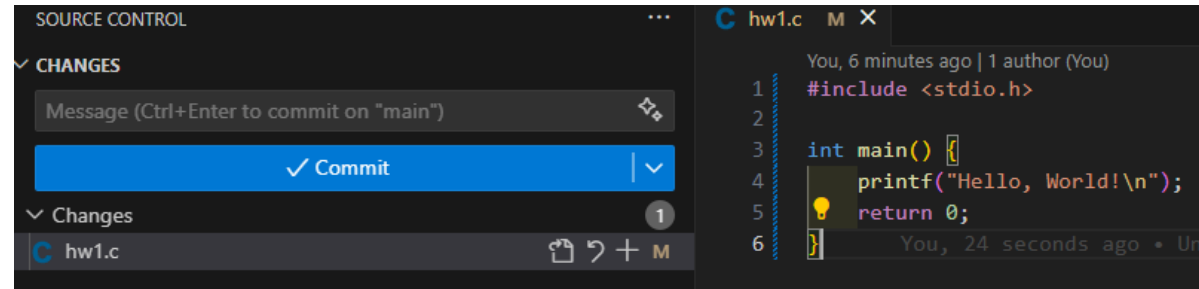
During development problems may arise; use diff to view differences between versions and quickly locate issues.

- diff: shows differences between two versions (or areas)
- Default comparison: Working directory vs Staging area (unstaged changes)
- Other comparisons: Staging area vs last commit / any two commits
- Purpose: pre-commit review for correctness; avoid committing leftover debug code



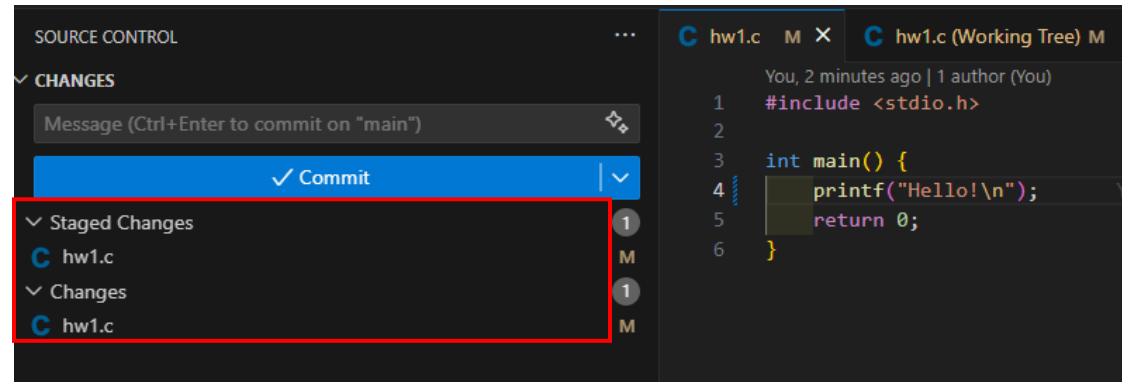
Using diff

Stage (add) some changes first.



Then make additional edits.

View the diff by clicking
the file or using a
command

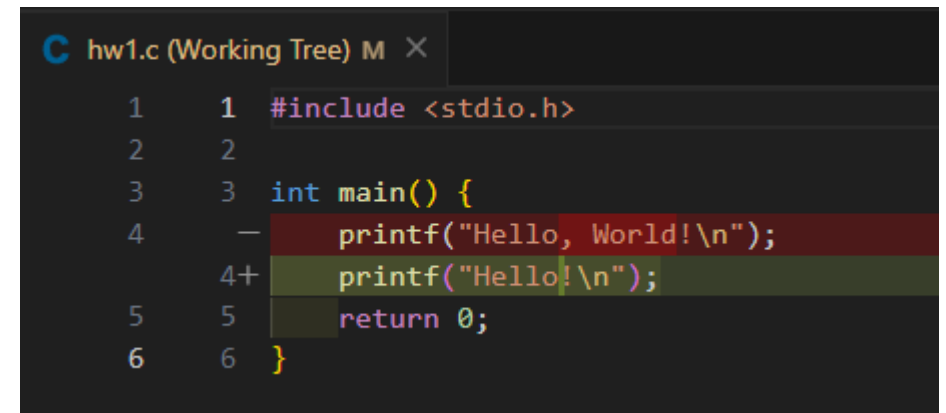


(showing the default
case here)

```
E:\GitExample>git diff
diff --git a/hw1.c b/hw1.c
index 8321e25..6134ec3 100644
--- a/hw1.c
+++ b/hw1.c
@@ -1,6 +1,6 @@
 #include <stdio.h>

int main() {
- printf("Hello, World!\n");
+ printf("Hello!\n");
  return 0;
}
\ No newline at end of file
```

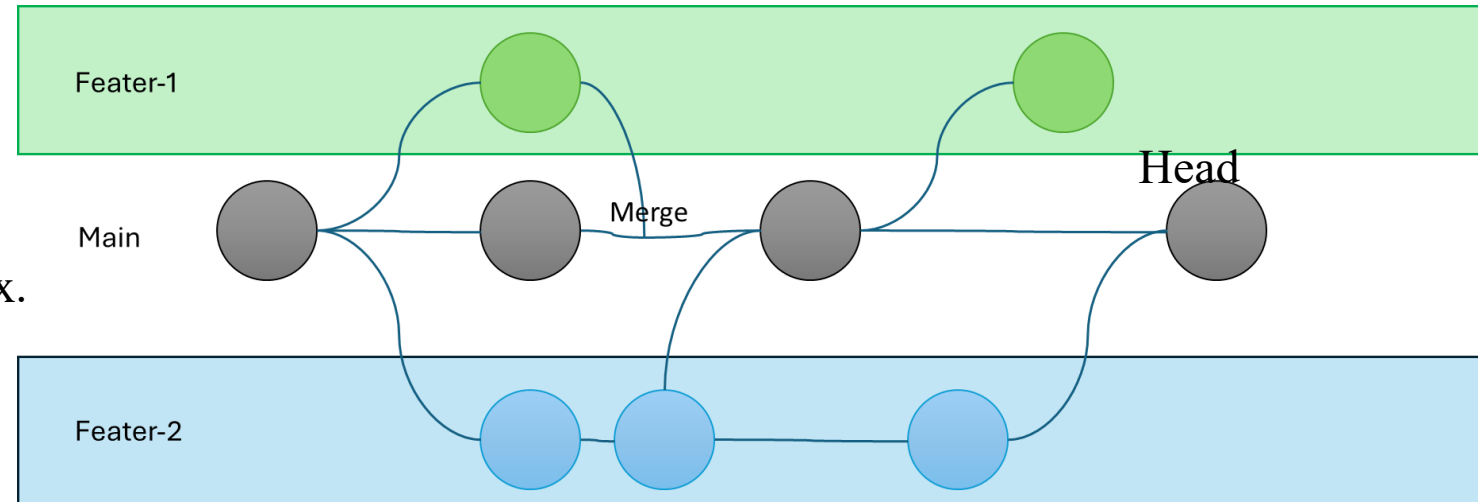
Output symbols:
Lines starting with -
indicate deletions.
Lines starting with +
indicate additions.



Branch

Branch Basics

- A branch is a lightweight pointer to a sequence of commits.
- Enables parallel development / experimentation without breaking the main line.
- main: stable / release-ready version.
- Feature branch: implements a single feature or fix.
- After merging back into main, the feature branch can be deleted.
- HEAD: your current location (can point anywhere).



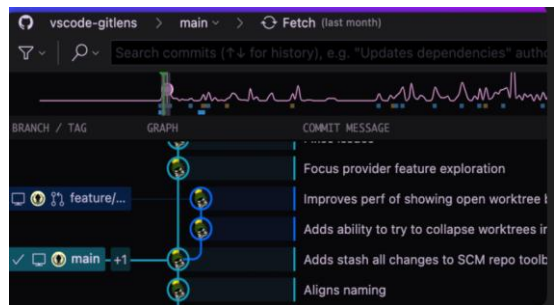
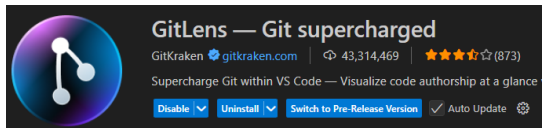
Create and View Branches

View all branches

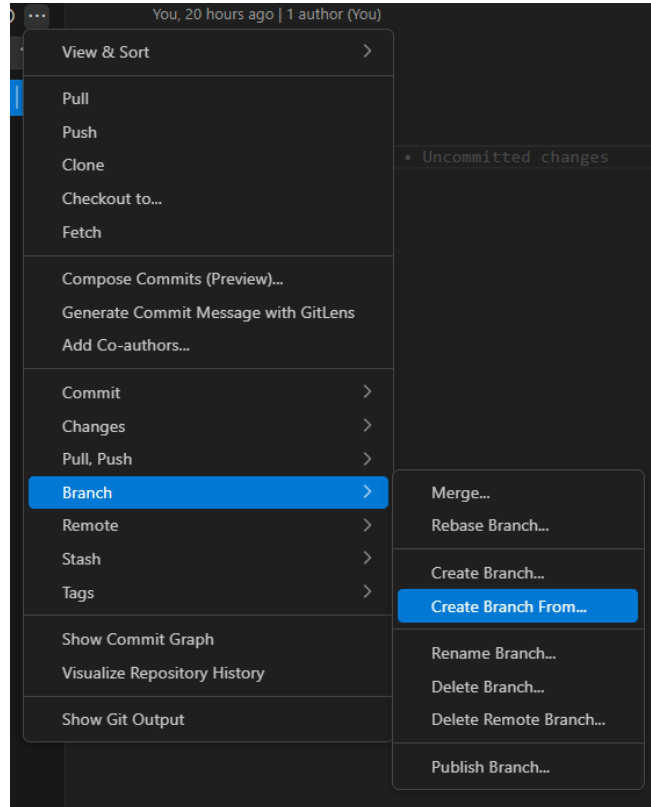
git branch

```
E:\GitExample>git branch
* main
```

Recommended: use visual tools like GitLens for inspection



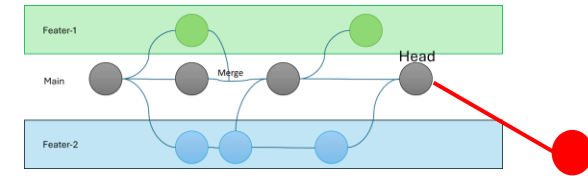
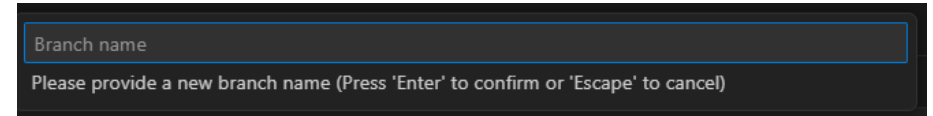
Create a new branch



git branch <BranchName >

Creates a new branch from the current position (current HEAD).

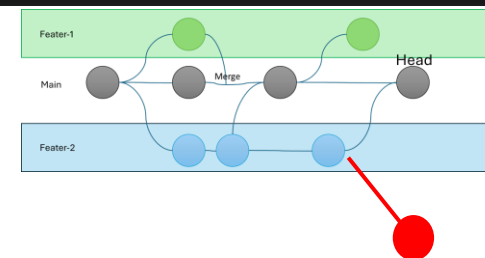
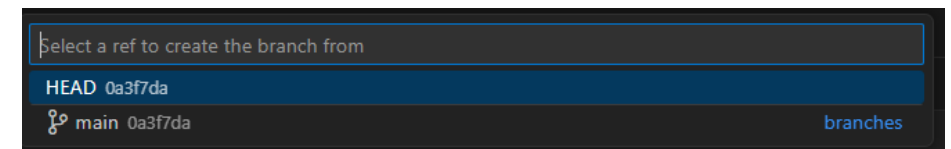
```
E:\GitExample>git branch feature-login
```



git branch <BranchName > <BaseRef>

Creates a new branch from the specified base (commit, tag, or another branch).

```
E:\GitExample>git branch feature-login 0a3f7dad895e65210a456a145090d631e310f91d
```



Switch Branches

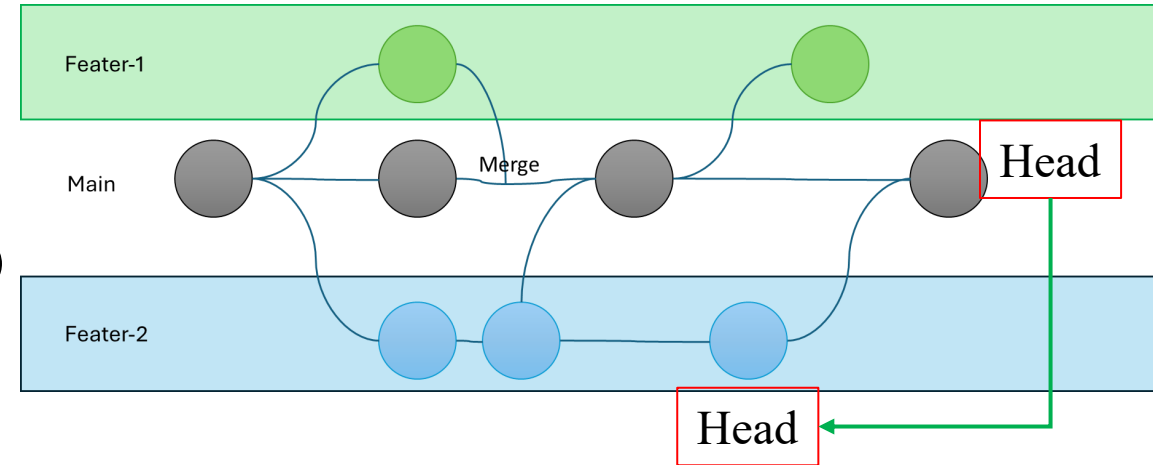
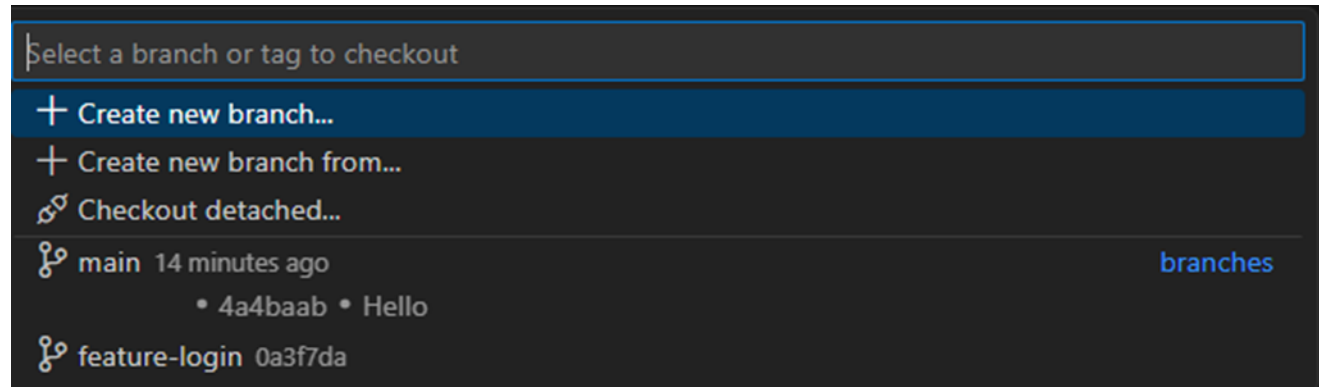
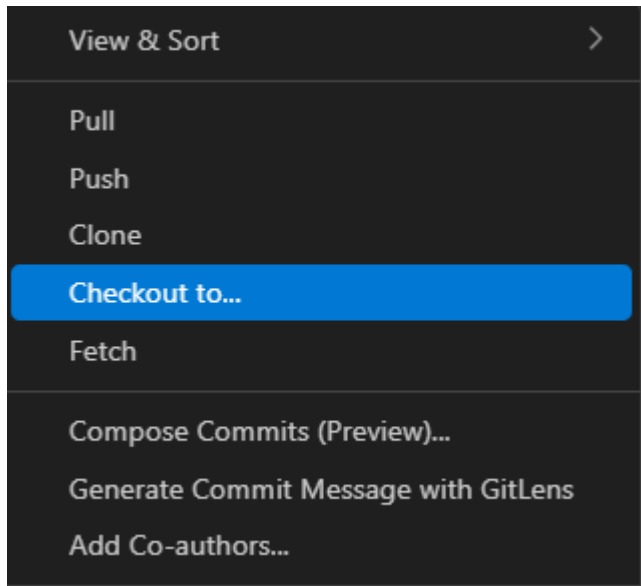
- `git switch main`
- `git checkout main` (older command, broader functionality)

CMD

```
E:\GitExample>git checkout feature-login
Switched to branch 'feature-login'

E:\GitExample>git switch feature-login
Already on 'feature-login'
```

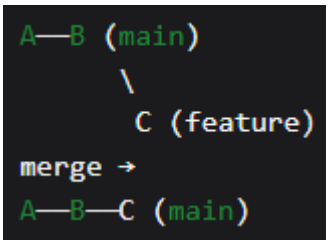
VS Code



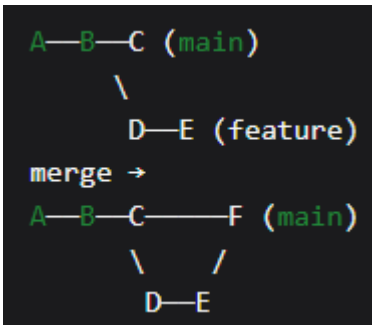
Merge Branches

After completing work on a branch, integrate it into another branch. Two scenarios:

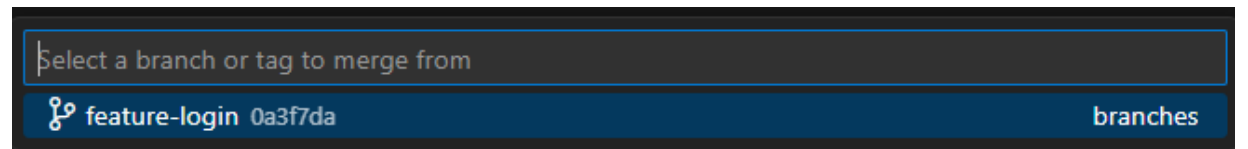
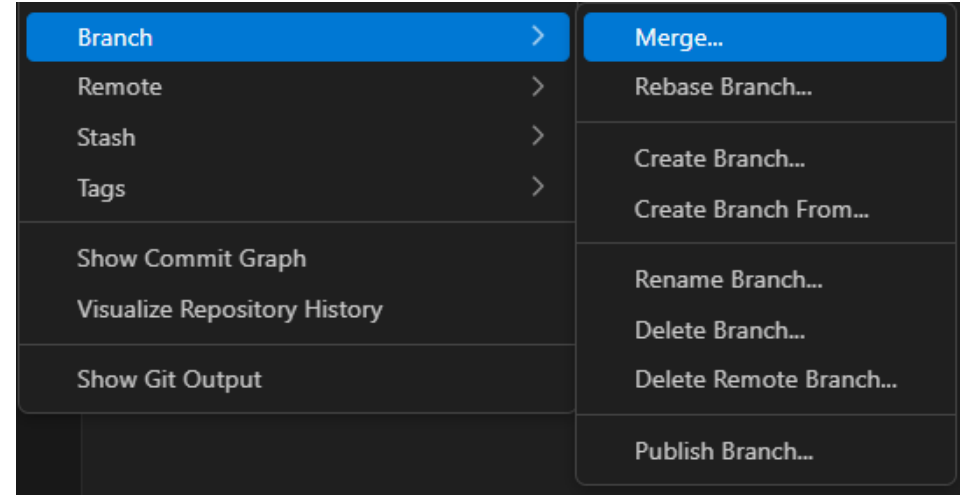
1. Fast-forward: main has not advanced $\rightarrow$ simply move the pointer forward.



2. Merge commit: both sides have advanced → create a new merge commit node.



Rebase is more complex; we will skip it for now.



Merge Workflow

Before merging: ensure main is

```
git switch main
git pull origin main
git merge feature-x
```

If the same line was changed on both sides → conflict.

Conflict markers

```
<<<<<< HEAD
Line from main
=====
Line from feature
>>>>>> feature-x
```

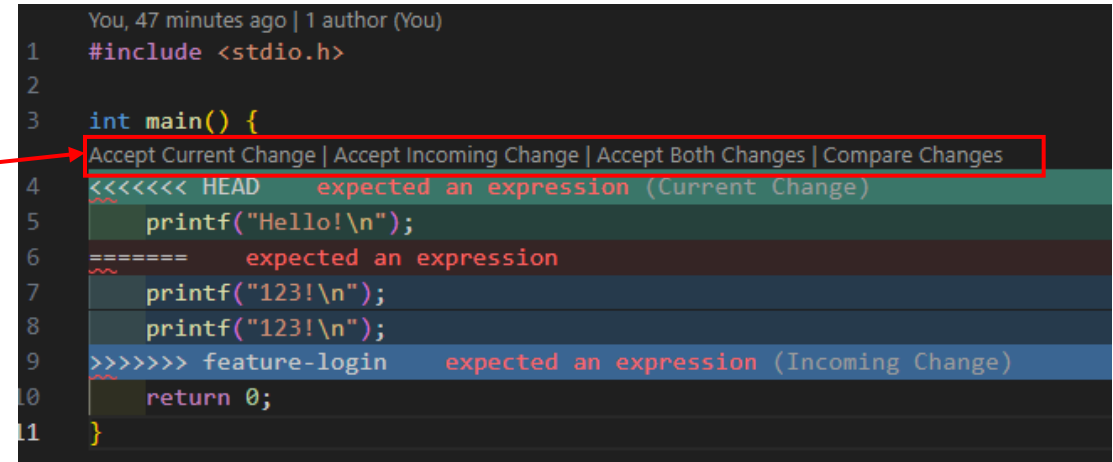
After resolving :

```
git add <file>
git commit
```

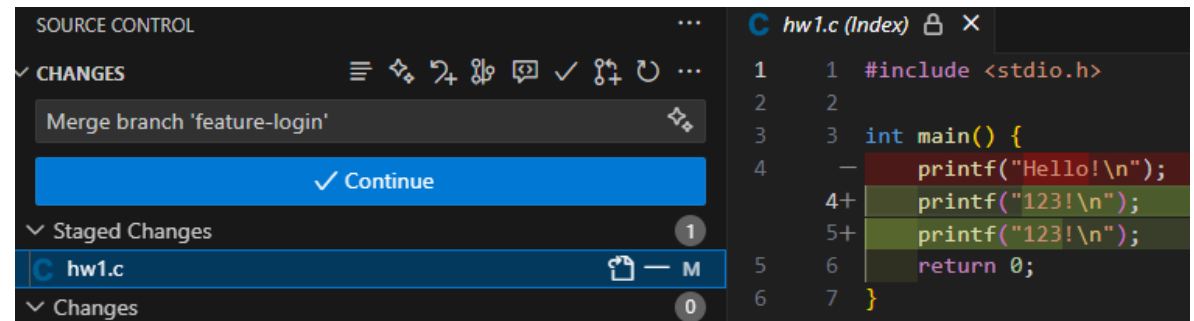
choose which version of each conflicting line to keep.

After the merge, review the diff and commit the update.

GUI



```
You, 47 minutes ago | 1 author (You)
1  #include <stdio.h>
2
3  int main() {
4  <<<<<< HEAD      expected an expression (Current Change)
5      printf("Hello!\n");
6  ===== expected an expression
7      printf("123!\n");
8      printf("123!\n");
9  >>>>>> feature-login expected an expression (Incoming Change)
10     return 0;
11 }
```



Cleanup and Delete Branches

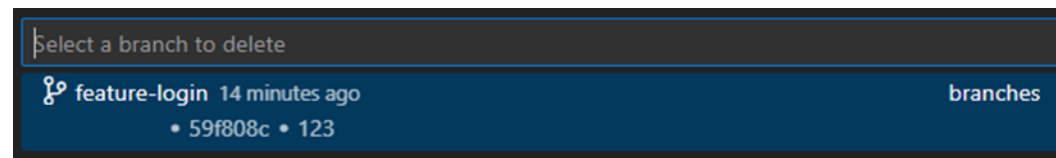
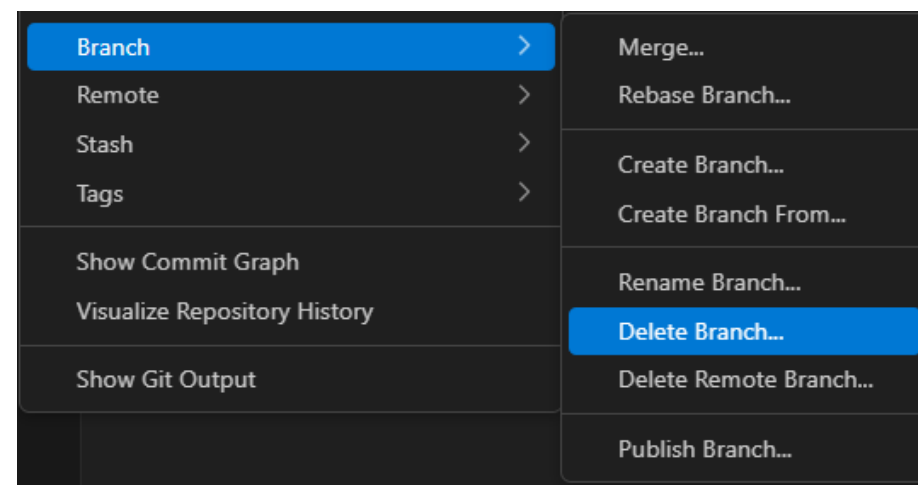
After merging, consider deleting branches you no longer need to avoid long-lived “zombie” branches.

Local deletion after merge

```
git branch -d feature-x    # 安全删除 (已合并)  
git branch -D feature-x    # 强制删除 (未合并, 慎用)
```

Remote deletion

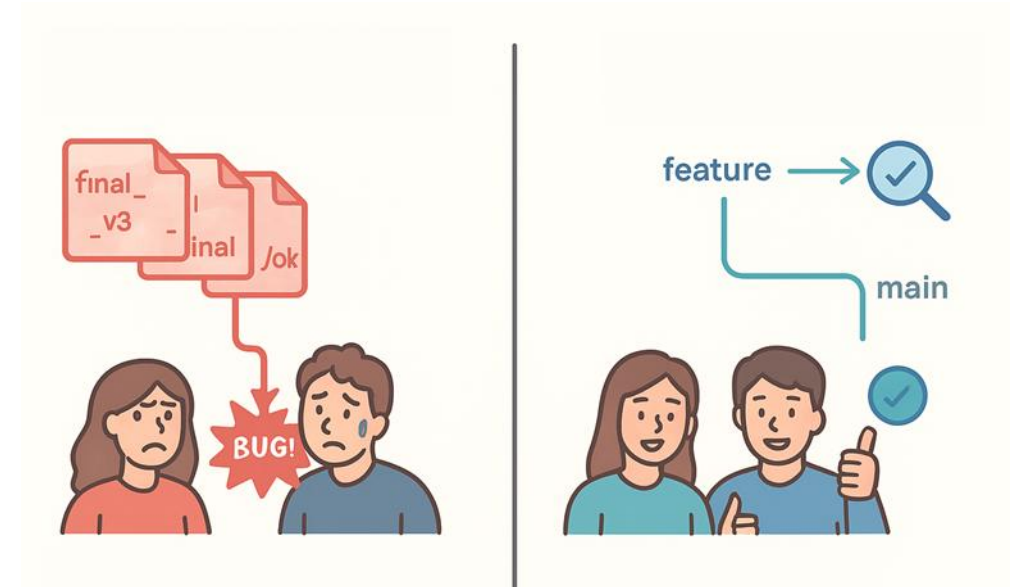
```
git push origin --delete feature-x
```



Workflow

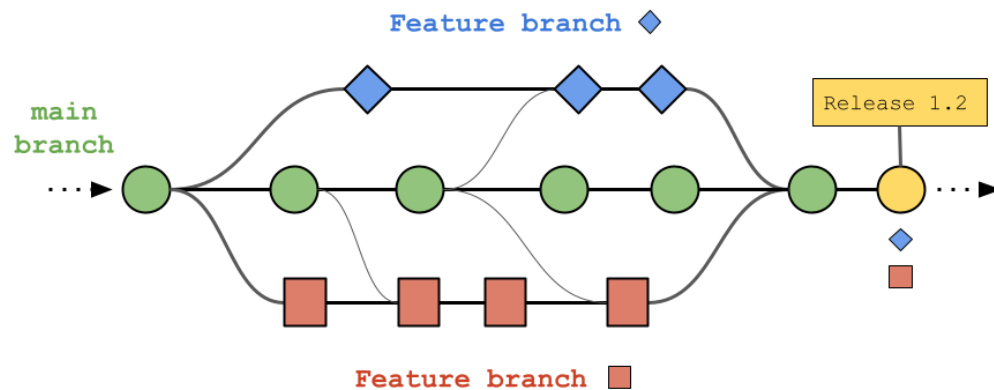
Why a Workflow?

- **Reduce conflicts:** develop each feature in an isolated branch
- **Clear ownership:** who does what is visible via branch names & PRs
- **Easier reviews:** PRs let peers/TAs check quality
- **Stable main:** keep main always runnable / releasable
- **Clean history:** every change has a clear description and record
- **Fast rollback:** can trace issues to a single feature branch
- **Build collaboration habits:** foundation for internships / capstone projects
- Avoid chaos of everyone overwriting “latest files”



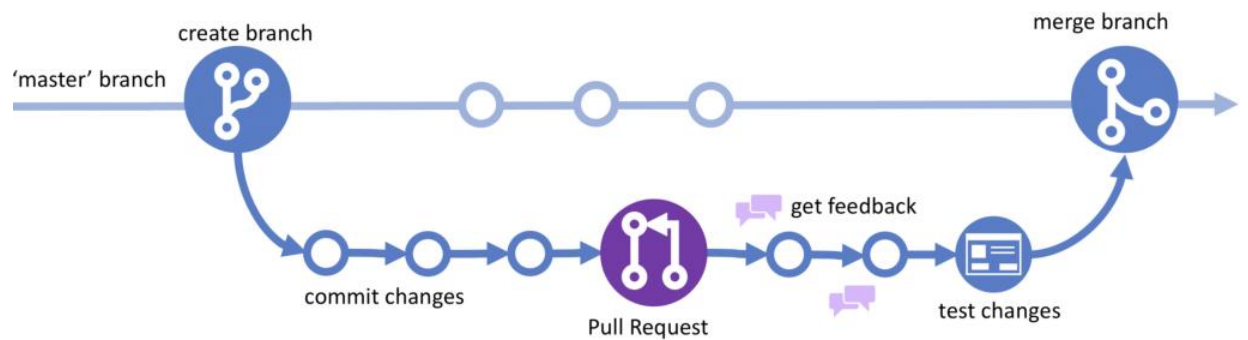
Workflow

- Solo: main + topic branches
- Common team: feature branches + pull requests
- Other models: GitHub Flow / Git Flow
- Each workflow defines its branch types; naming pattern: BranchType/name



[How to stop using feature branches - codefortynine GmbH](#)

GitHub Flow



Copyright © 2018 Build Azure LLC

<http://buildazure.com>

[Introduction To GitHub And Git Version Control Workflow | Build5Nines](#)

Feature Branch Workflow

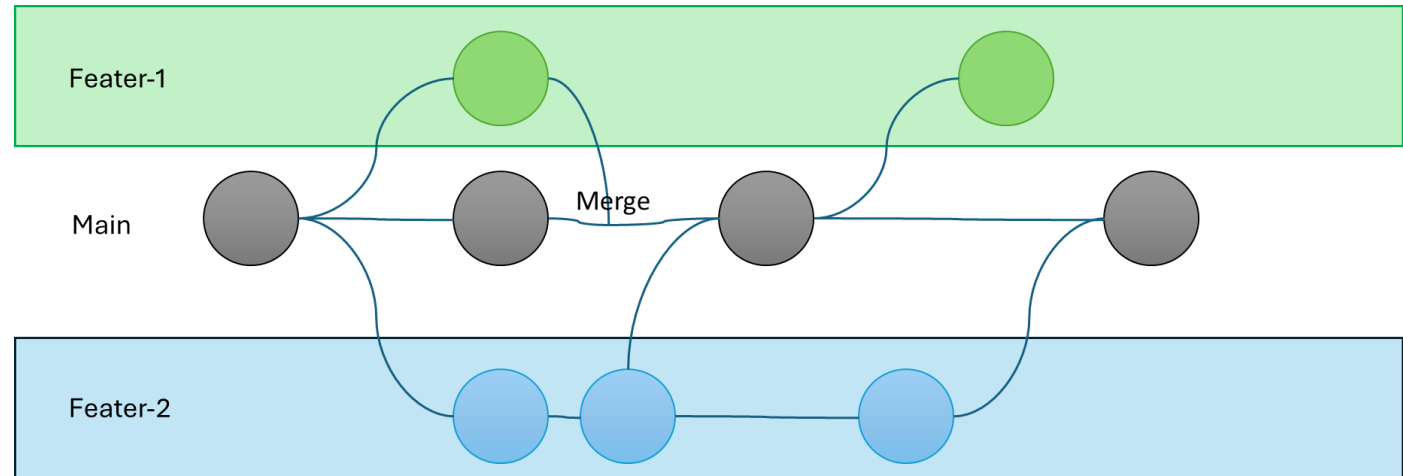
- Simplest and scalable: main + multiple feature branches
- Each feature / fix = a short-lived branch
- Keep main always runnable / releasable
- Great starting point for beginners / solo development

Branch naming guidelines:

All lowercase + / or -

Use a clear verb or noun phrase; avoid vague terms

(bad: update, good: feature/pdf-export)

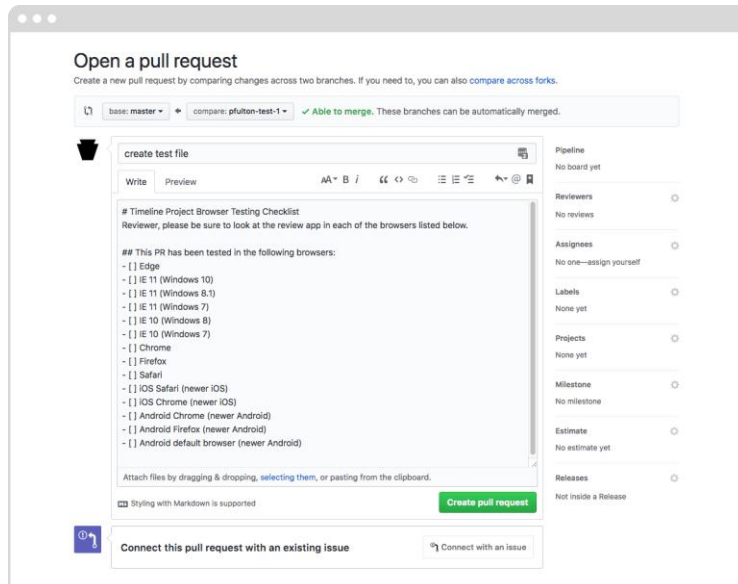


Github flow

- Suitable for team collaboration
- Only one long-lived branch: master (main)
- All other branches are short-lived feature branches
- Merge = deploy; keep trunk always releasable

Pull Request (PR) purposes

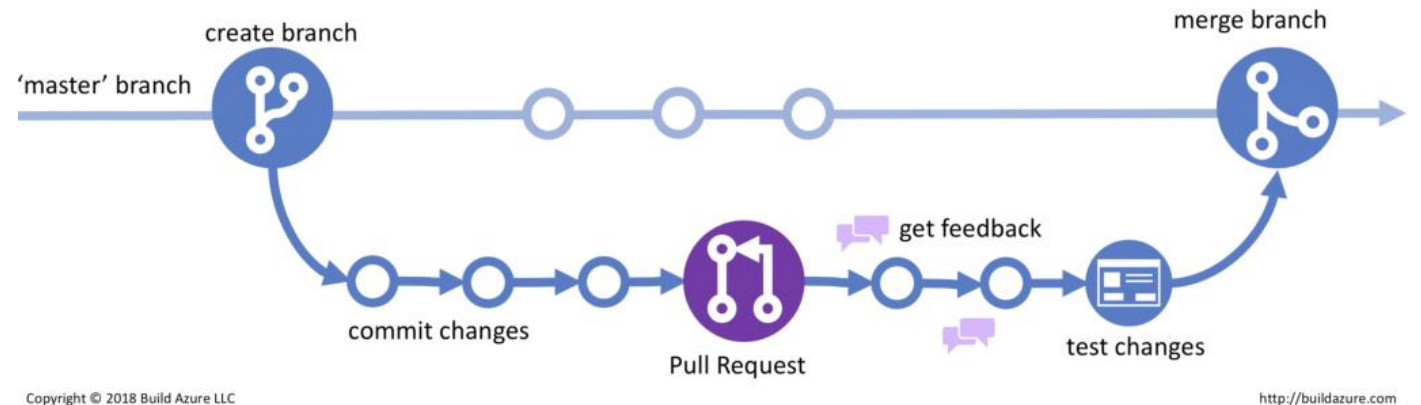
- Contextual change proposal
- Provides: What (summary), Why (reason), How to test
- Serves as: code review entry, discussion log, decision & accountability trail



Flow steps

1. Branch off master (feature/login, fix/timeout)
2. Commit in small increments and push frequently; open a PR when discussion is needed or core work is done
3. PR undergoes discussion / code review / automated tests (CI)
4. After approval, merge back to master (often fast-forward or squash)
5. Deploy latest master; delete merged branch to keep repo tidy

GitHub Flow



[Introduction To GitHub And Git Version Control Workflow | Build5Nines](#)

Git flow

Used in larger teams: Git Flow defines five branch types: master, develop, hotfix, release, feature

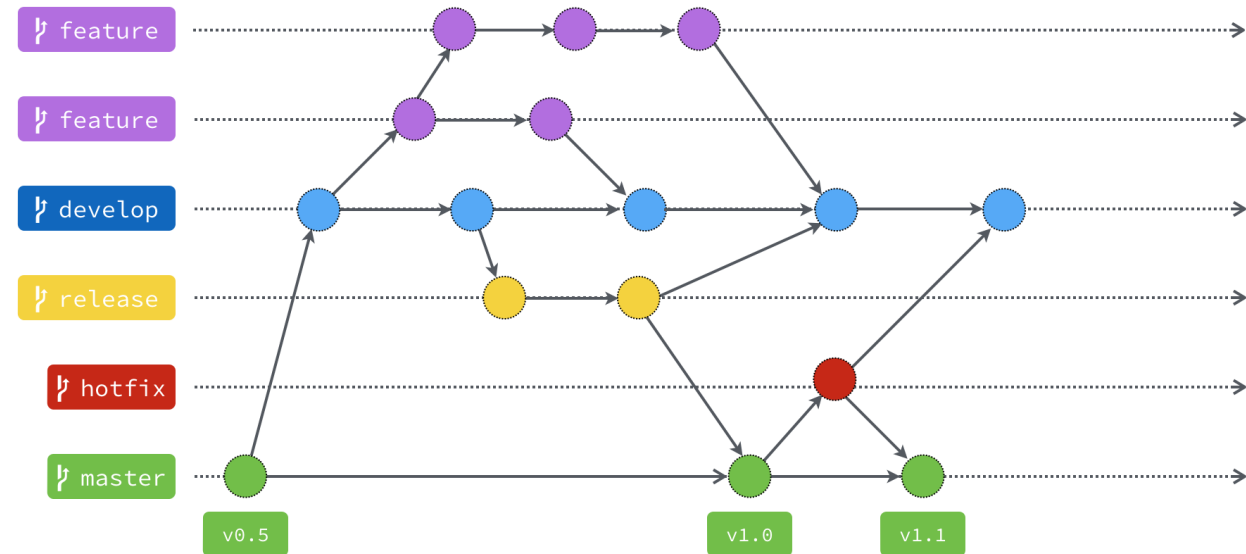
Master: stable, always deployable (same role as earlier main)

Develop: primary development/integration branch; base for short-lived branches

Hotfix: urgent production fixes

Release: pre-release branch cut from develop to stabilize & test before merging to master

Feature: branch for adding new functionality

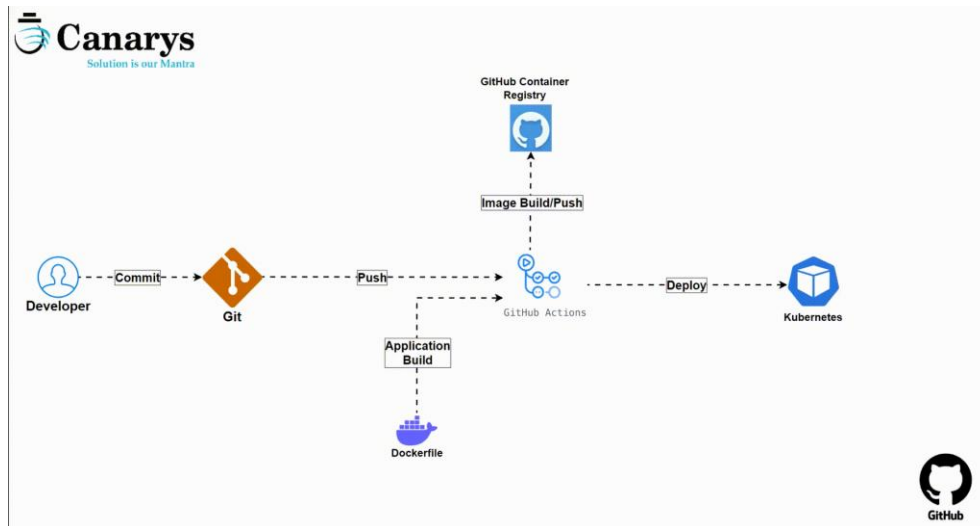
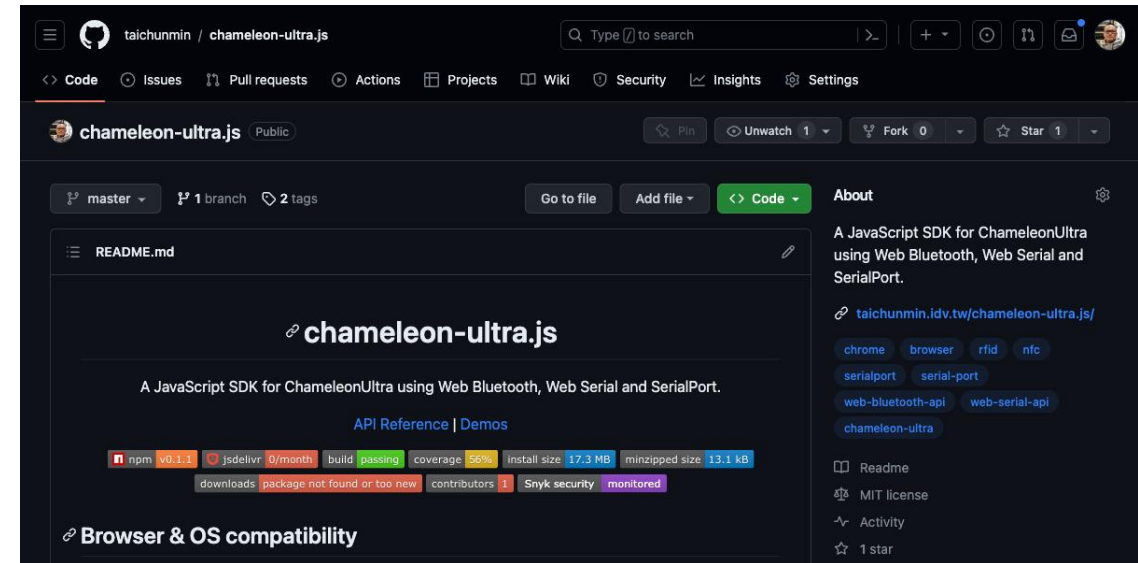


Git Flow 是什麼？為什麼需要這種東西？ - 為你自己學 Git | 高見龍

GitHub

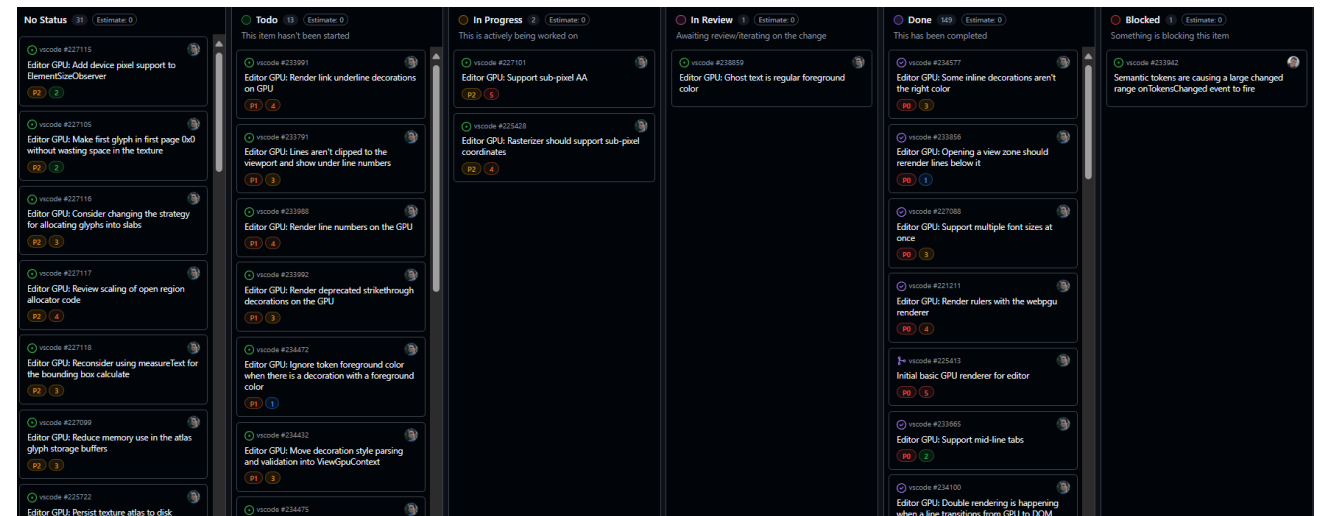
Managing Your GitHub

- Not only commands—learn to manage a project
- Clear docs = lower communication cost
- Conventions & workflows = less friction
- Automation = focus on core code



Streamline Your Docker Workflow with GitHub Actions - Canarys

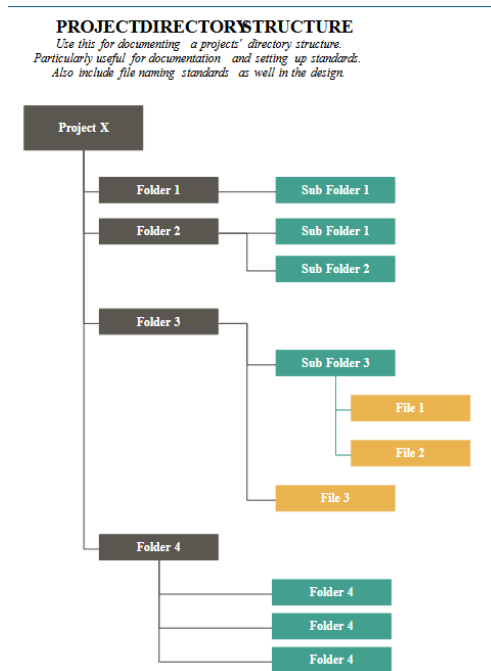
Day 24 README.md - iT



Backlog · VS Code Editor GPU Renderer

Repository Structure & Naming

- Short, clear names (e.g. led-controller, signal-analyzer)
- README: project purpose + quick start
- Layered directories: src/ docs/ tests/ examples/
- Begin structuring once you need more than a single file
- Avoid committing large archive files directly (use Releases or external storage)



[Project Directory Structure | Creately](#)

📁 .config	chore: update baselines (#263626)	2 days ago
📁 .devcontainer	fix: changed the node js version from 20 to 22 as per #25719...	last month
📁 .eslint-plugin-local	Update target for build and test scripts	last month
📁 .github	Reduce inclusion of telemetry instructions (#263685)	2 days ago
📁 .vscode	Update notebook milestones (#263172)	5 days ago
📁 build	Extract more types for reuse (#264032)	10 hours ago
📁 cli	Fix --commit-id flag for code serve-web (#258904)	29 days ago
📁 extensions	MSAL Redirect Funkiness (#264057)	8 hours ago
📁 remote	chore: update electron@37.3.1 (#263552)	3 days ago
📁 resources	fix: switch everyone to DEB822 (#260171)	3 weeks ago
📁 scripts	Allow to hide Copilot via user setting and policy (fix #24961...	2 weeks ago
📁 src	Fix disposal of chat session renderer (#264104)	2 hours ago
📁 test	Have the lifecycle dependent on the Page since we only sup...	3 hours ago
📄 .editorconfig	No forcing tabsiz on users	7 years ago
📄 .eslint-ignore	Fix eslint not linting our custom eslint rules	5 months ago
📄 .git-blame-ignore-revs	eng - add ESM migration commit to .git-blame-ignore-revs...	last year
📄 .gitattributes	Whitelist comments in all JSON files. For #129206	4 years ago

[microsoft/vscode: Visual Studio Code](#)

README Essentials

Entry point: let readers quickly understand what it does, how to use it, and the license terms

- Project overview
- Installation & run steps
- Examples
- Directory structure
- Contribution guidelines
- License

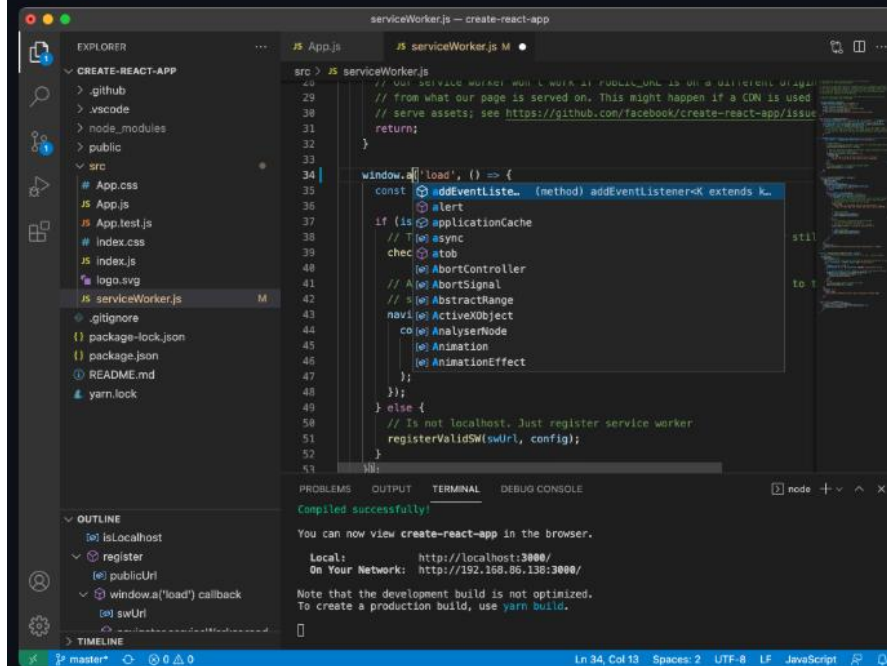
Visual Studio Code - Open Source ("Code - OSS")

feature-request issues 3.4k open bug issues 3.6k open chat on gitter

The Repository

This repository ("code - oss") is where we (Microsoft) develop the [Visual Studio Code](#) product together with the community. Not only do we work on code and issues here, we also publish our [roadmap](#), [monthly iteration plans](#), and our [endgame plans](#). This source code is available to everyone under the standard [MIT license](#).

Visual Studio Code



Related Projects

Many of the core components and extensions to VS Code live in their own repositories on GitHub. For example, the [node debug adapter](#) and the [mono debug adapter](#) repositories are separate from each other. For a complete list, please visit the [Related Projects](#) page on our [wiki](#).

Bundled Extensions

VS Code includes a set of built-in extensions located in the [extensions](#) folder, including grammars and snippets for many languages. Extensions that provide rich language support (code completion, Go to Definition) for a language have the suffix `language-features`. For example, the `json` extension provides coloring for `JSON` and the `json-language-features` extension provides rich language support for `JSON`.

Development Container

This repository includes a Visual Studio Code Dev Containers / GitHub Codespaces development container.

- For [Dev Containers](#), use the [Dev Containers: Clone Repository in Container Volume...](#) command which creates a Docker volume for better disk I/O on macOS and Windows.
 - If you already have VS Code and Docker installed, you can also click [here](#) to get started. This will cause VS Code to automatically install the Dev Containers extension if needed, clone the source code into a container volume, and spin up a dev container for use.
- For Codespaces, install the [GitHub Codespaces](#) extension in VS Code, and use the [Codespaces: Create New Codespace](#) command.

Docker / the Codespace should have at least 4 Cores and 6 GB of RAM (8 GB recommended) to run full build. See the [development container README](#) for more information.

Code of Conduct

This project has adopted the [Microsoft Open Source Code of Conduct](#). For more information see the [Code of Conduct FAQ](#) or contact opencode@microsoft.com with any additional questions or comments.

License

Copyright (c) Microsoft Corporation. All rights reserved.

Licensed under the [MIT](#) license.

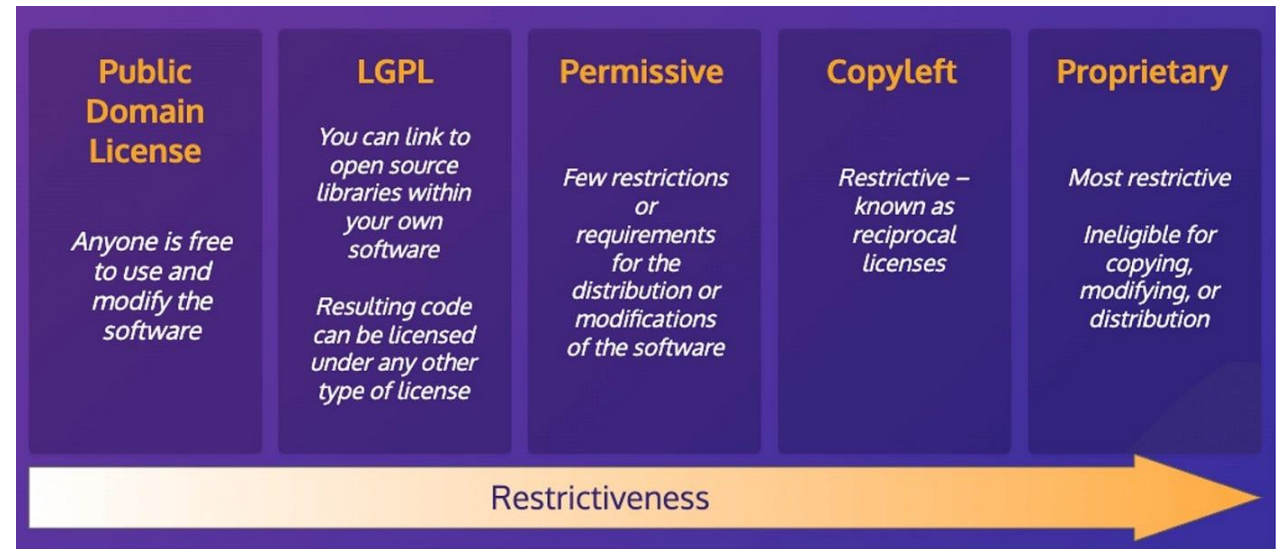
About LICENSE

When building a project, also consider legal aspects to protect your rights.

Add a LICENSE file so others know how you permit use.

- No license = default restrictive (others cannot freely use)
- Common choices: MIT (permissive), Apache 2.0 (adds patent terms), GPL (copyleft)
- Classroom / learning projects: MIT is often a good fit
- Place a file named LICENSE at the repository root

6 Common Open Source Licenses Explained



Software Licensing. What are Software Licenses? | by Udesch Indumina | Medium

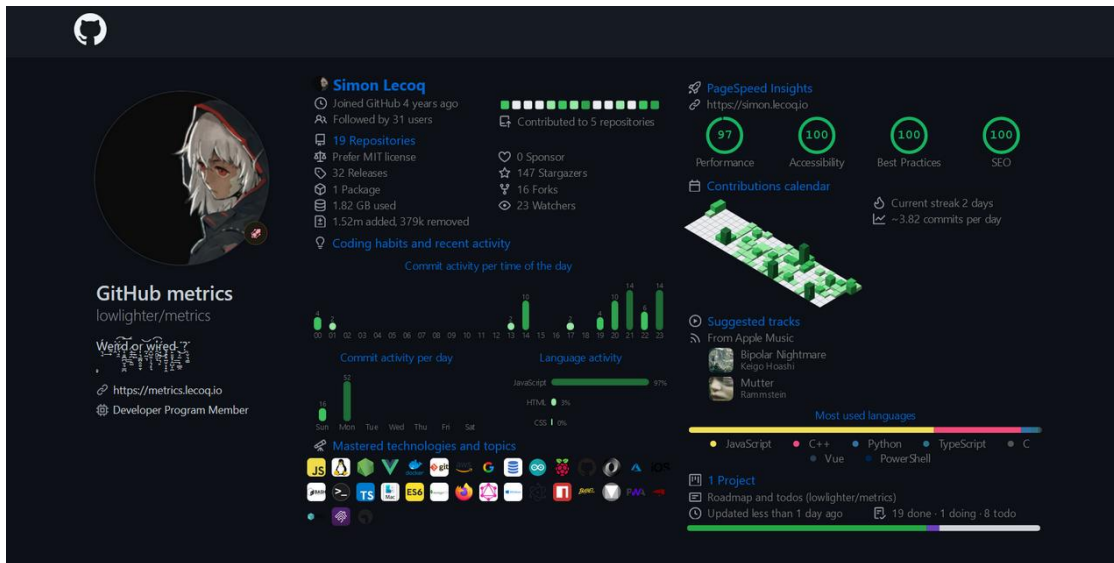
snky	Copyleft					Permissive			
	GPL	AGPL	LGPL	OSI	MPL	Apache	MIT	BSD	Unlicense
Permissions in addition to commercial use, distribution, modification:									
Patent use	●	●	●	●	●	●	●	●	●
Patent use	●	●	●	●	●	●	●	●	●
Conditions									
Disclose source	●	●	●	●	●	●	●	●	●
License & copyright notice	●	●	●	●	●	●	●	Source	●
Network use is distribution	●	●	●	●	●	●	●	●	●
Same license	●	●	Library	●	File	●	●	●	●
State changes	●	●	●	Some	●	●	●	●	●
Limitations/Disclaimers									
Liability	●	●	●	●	●	●	●	●	●
Warranty	●	●	●	●	●	●	●	●	●
Trademark use	No explicit limitation				●	●	●	●	●

Open Source Licenses: Types and Comparison | Snky

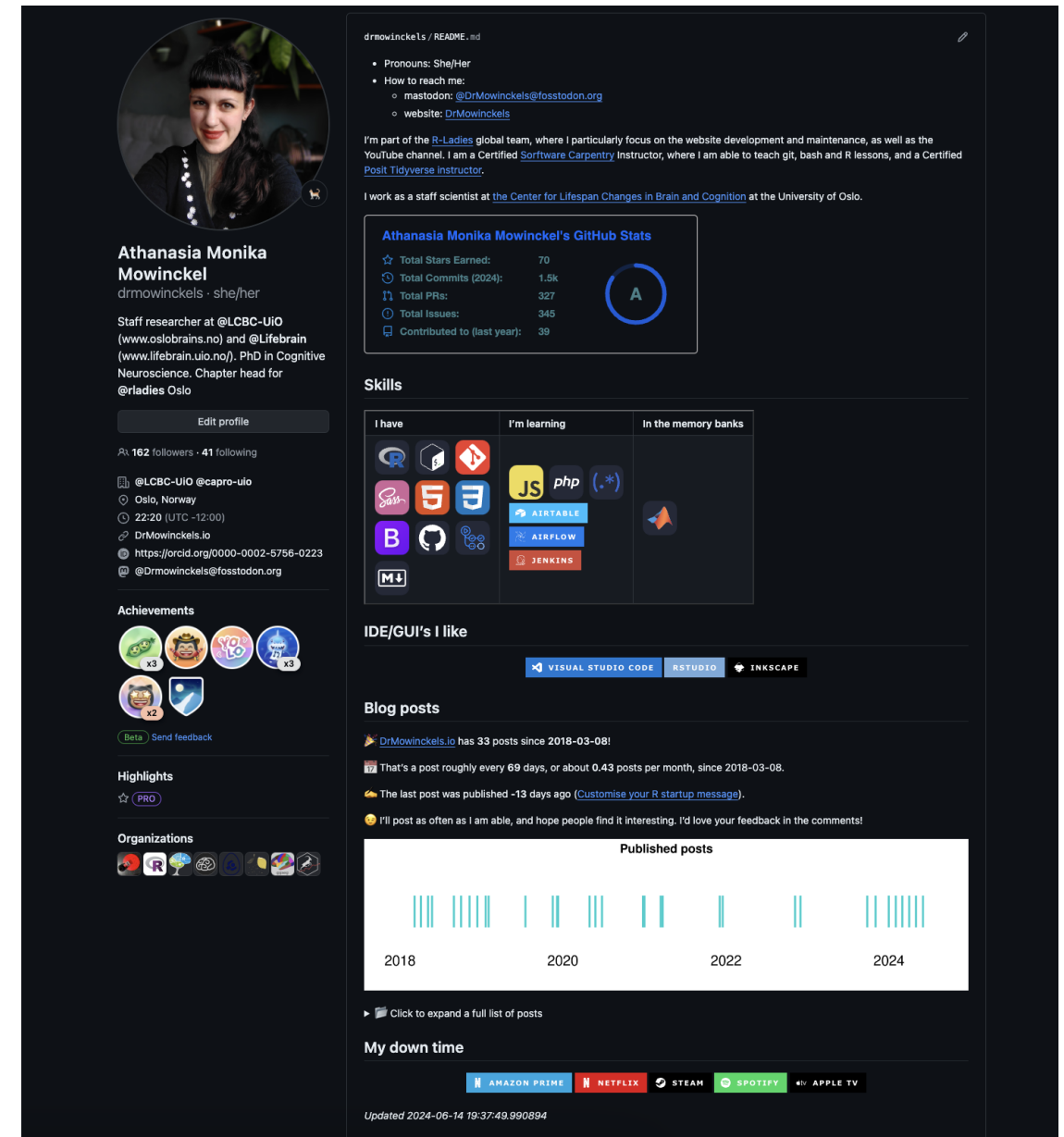
GitHub Profile Management

Why Optimize Your Profile

- First impression: your technical résumé landing page
- Attracts admissions, internship, and collaboration opportunities
- Demonstrates implementation depth and learning cadence
- Clear structure saves others' evaluation time
- Aim for Clarity + Credibility, not flashy clutter



[lowlighter/metrics](#):  An infographics generator with 30+ plugins and 300+ options to display stats about your GitHub account and render them as SVG, Markdown, PDF or JSON!



[Improving your GitHub Profile - Dr. Mowinckel's](#)

Create Profile README

- Create a special repository: name it exactly your GitHub username
- Check “Initialize with README”
- The README.md content becomes your profile homepage
- You can use a template to speed up creation

Hi, I'm [YOUR_NAME] 🙋

Tagline

[One-line positioning statement, e.g.: Student firmware developer focused on low-power MCUs & reliability testing]

About Me

- Current focus: [e.g., sensor aggregation / RTOS scheduling / frontend data viz]
- Highlight: [e.g., Set up automated firmware test pipeline cutting manual checks by 30%]
- Motivation: [e.g., Enjoy validating hardware behavior via scripted measurement]
- Looking for: [e.g., Embedded internship / OSS driver collaboration]

Tech Stack

Languages: C (proficient), Python (proficient), Rust (learning)
Platforms / Frameworks: STM32 HAL, ESP-IDF, FreeRTOS
Tools: Git, GitHub Actions, PlatformIO, Docker
Testing / Quality: Unity C Test, pytest, clang-format
Learning: Rust embedded, hardware-in-the-loop testing

Featured Projects

1. [PROJECT_NAME_1]

Value: [e.g., Modular I2C sensor hub with auto-detection & unified output]
Tech: C, FreeRTOS, I2C, PlatformIO
Role: [e.g., System architecture + device abstraction layer]
Highlight: [e.g., Reduced code changes needed for new sensor by 50%]
Next: [e.g., Add offline caching & coverage report]

2. [PROJECT_NAME_2]

Value:
Tech:
Role:
Highlight:
Next:

3. [PROJECT_NAME_3]

Value:
Tech:
Role:
Highlight:

Learning Focus

- Topic: [e.g., Low-power state transitions & current profiling]
- Approach: [e.g., Instrumentation with current probe + scripted logging]
- Result (so far): [e.g., Reduced sleep current from 42µA to <15µA]

Recent Updates

- 2025-08: Implemented auto registration table for sensors
- 2025-07: Added GitHub Actions unit test workflow
- 2025-07: Published note on RTOS jitter measurement

Experiments (Optional)

- [Experiment]: Hypothesis → Method → Observation
- Example: Interrupt priority tuning reduced average latency by 18%

Open Source Contributions (Optional)

- PR: #1234 to repoX fixing memory release
- Issue: #56 Documentation correction + patch
- Plan: Seek first-timers-only issues monthly

Posts / Notes (Optional)

- [RTOS Task Timing Measurement](#) – GPIO tagging + counter method
- [I2C Abstraction Design Notes](#)

Workflows (Optional)

- Commit strategy: small, prefixed (feat, fix, refactor, test)
- Testing: CI on push, manual HIL cycle weekly
- Quality: clang-format / static analysis (cppcheck)

Contact

- Email: yourname (at) example.com
- LinkedIn: [...]in/your-id
- Other (optional): personal site / blog

Extras

build repo or workflow not found license repo not found

(Optional) Dynamic Sections

Dynamic Stats & Widgets

[beautify-github-profile](#) This is a curated profile-writing resource containing numerous practical tools and templates.

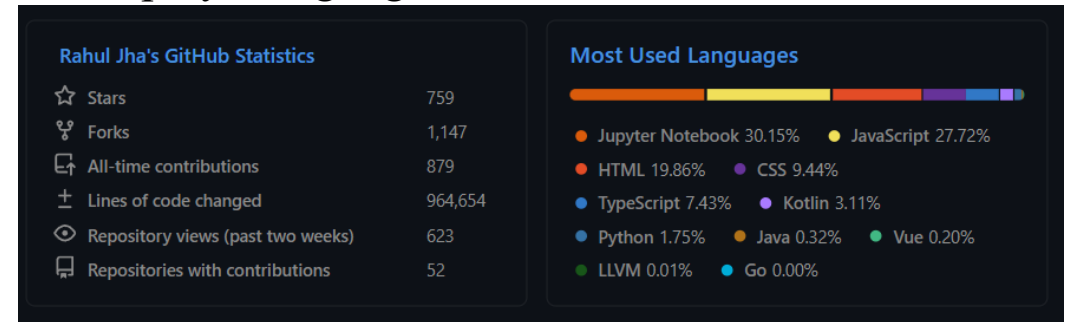
Displays languages used and related metrics

Hello friends 🖐️

How are you? If you want to make the look of your github profile more beautiful, you have come to the right place.

📖 Table of Contents

- 🔗 [The first step : Set up the GitHub Repository](#)
- 💡 [Where do we get ideas ?](#)
- 🚩 [What do we do after seeing these profiles ?](#)
 - 🌸 [Badges](#)
 - 🛠️ [Widgets](#)
 - ✅ [Icons](#)
 - ⚙️ [Profile Generator](#)
 - 😊 [Emojis](#)
- ⭐ [Give A Star](#)



Shows activity level and engagement timeline

